

claude-windows / 56b7f849-5d56-4770-93e0-b3b4da943700

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\56b7f849-5d56-4770-93e0-b3b4da943700\subagents\agent-a68b023d0580bce46.jsonl`  
- SHA-256: `11ad3c34546724360d9356c6b406faacceb211d148b128cee0da0982f674bdf8`  
- Source modified: `2026-06-07T06:43:26+00:00`  
- Imported at: `2026-07-05T16:48:27+00:00`  
- project: `subagents`  
- session_id: `56b7f849-5d56-4770-93e0-b3b4da943700`
```

Transcript

1. user (2026-06-07T06:41:39.775Z)

Explore the repository at C:\Users\avidu\Projects\sports-data-collector (a Python "sports-data-collector" CLI). I need a thorough map of its processing pipeline and every point where it writes output to disk, so I can add a rich per-video "observability report" feature.

Please report back, with file_path:line references:

1. ****CLI entrypoints****: The main CLI is `src/cli/curate.py`. List every subcommand (argparse subparsers / commands) it exposes, and one-line what each does. Note especially `import-local`, `export`, `validate-delivery`, `convert-cvat` and any ingestion commands.
2. ****Per-video processing stages****: For the ingestion/curation path (e.g. `import-local`, `convert-cvat`), trace the stages a video/annotation goes through end-to-end ? parsing, validation, canonicalization, registration into any DB/store, export. Name the key functions and the files they live in.
3. ****Data model / store****: How does it persist state? Is there a SQLite DB, a JSON store, CSV files, a manifest? Where (file paths) and what's the schema/shape? List the key fields tracked per video/rally.
4. ****Existing outputs****: Enumerate every artifact the tool currently writes ? report files (e.g. `\_report.md`/`.json`), review CSVs, exported CSVs, manifests. For each, the producing function (file:line) and the shape/columns.
5. ****Validation****: How does `validate-delivery` work ? what checks does it run, what does its report contain, and what function assembles that report? (look for gap checks / possible_missed_rally / blocker logic)
6. ****Canonical data shapes****: The canonical rally/segment record shape, the sport profile abstraction, and any fps/timestamp handling (seconds vs mm:ss, frame/fps recompute).
7. ****Tests****: List the test files and which module each covers, so I know where to add tests for a new report feature.

Be thorough and cite exact file paths and line numbers. I want enough detail to design where a new "rich observability report per processed video" emitter should hook in, and how to keep it consistent with existing report-writing patterns.

2. assistant (2026-06-07T06:41:41.502Z)

I'll conduct a thorough exploration of the sports-data-collector repository to map its pipeline and output mechanisms. Let me start by understanding the overall structure.

3. user (2026-06-07T06:41:42.839Z)

```
./cleanup.py
```

```
./src/cli/curate.py
./src/cli/ingest.py
./src/core/config.py
./src/core/db.py
./src/core/downloader.py
./src/core/parsing.py
./src/core/schemas/badminton.py
./src/core/schemas/base.py
./src/core/schemas/cricket.py
./src/core/schemas/pickleball.py
./src/core/schemas/tabletennis.py
./src/core/schemas/tennis.py
./src/core/validator.py
./src/crawlers/badminton_crawler.py
./src/parsers/badminton/coachai.py
./src/parsers/base_parser.py
./src/parsers/cvat/rally_cvat.py
./src/parsers/cvat/__init__.py
./tests/test_cli.py
```

4. assistant (2026-06-07T06:41:44.112Z)

Let me get a broader overview of the repository structure.

5. user (2026-06-07T06:41:44.473Z)

```
total 105
drwxr-xr-x 1 avidu 197609    0 Jun  6 20:22 .
drwxr-xr-x 1 avidu 197609    0 Jun  7 06:57 ..
-rw-r--r-- 1 avidu 197609 53248 Jun  2 22:37 .coverage
drwxr-xr-x 1 avidu 197609    0 Jun  6 19:46 .git
-rw-r--r-- 1 avidu 197609   264 May 26 01:26 .gitattributes
-rw-r--r-- 1 avidu 197609   897 Jun  2 22:36 .gitignore
drwxr-xr-x 1 avidu 197609    0 May 25 23:59 .pytest_cache
drwxr-xr-x 1 avidu 197609    0 Jun  6 20:22 .sixth
-rw-r--r-- 1 avidu 197609   9583 Jun  2 22:52 README.md
drwxr-xr-x 1 avidu 197609    0 Jun  2 22:30 __pycache__
-rwxr-xr-x 1 avidu 197609   6802 Jun  2 22:36 cleanup.py
drwxr-xr-x 1 avidu 197609    0 May 26 01:31 config
drwxr-xr-x 1 avidu 197609    0 Jun  6 15:46 data
drwxr-xr-x 1 avidu 197609    0 Jun  6 19:46 docs
-rw-r--r-- 1 avidu 197609    762 Jun  2 22:36 pyproject.toml
drwxr-xr-x 1 avidu 197609    0 May 26 00:10 src
drwxr-xr-x 1 avidu 197609    0 Jun  6 19:46 tests
```

6. assistant (2026-06-07T06:41:45.747Z)

Now let me read the main CLI file to understand the subcommands:

7. user (2026-06-07T06:41:46.199Z)

```
1     import os
2     import sys
3     import time
4     import json
5     import csv
6     import argparse
7     from typing import List, Dict, Any, Tuple
8     from tabulate import tabulate
9
10    from src.core.config import load_config
11    from src.core.parsing import safe_int as _safe_int, safe_float_required as _safe_float
12    from src.core.db import SportsDatabase
13    from src.core.schemas.base import VideoMetadata, SportType, LicenseType, CurationStatus
```

```

14     from src.core.schemas.badminton import BadmintonRally
15     from src.core.schemas.tennis import TennisPoint
16     from src.core.schemas.cricket import CricketDelivery
17     from src.core.schemas.pickleball import PickleballRally
18     from src.core.schemas.tabletennis import TableTennisRally
19     from src.core.validator import DatasetValidator
20     from src.core.downloader import download_best_video, probe_resolution_fps,
height_to_label
21
22
23     class SportsDataCLI:
24         def __init__(self, config_path: str = "config/config.yaml"):
25             self.config_path = config_path
26             self.config = self._load_config()
27
28             # Resolve DB path
29             db_path = self.config.get("database_path", "data/sports_metadata.db")
30             self.db = SportsDatabase(db_path)
31             self.validator = DatasetValidator(config_path)
32
33         def _load_config(self) -> Dict[str, Any]:
34             if not os.path.exists(self.config_path):
35                 print(f"Warning: Configuration file not found at {self.config_path}. Using
defaults.")
36             return load_config(self.config_path)
37
38         def status(self):
39             """Displays status summary of all videos in the database."""
40             # Query total count by status via the public DB read API.
41             rows = self.db.get_status_counts() # list of (sport, status, cnt)
42
43             if not rows:
44                 print("\nDatabase is currently empty. Run crawlers or import local data.\n")
45                 return
46
47             data = [[sport, status, cnt] for sport, status, cnt in rows]
48             print("\n=== Dataset Ingestion Status ===")
49             print(tabulate(data, headers=["Sport", "Status", "Count"], tablefmt="grid"))
50             print()
51
52         def import_local(self, args):
53             """Manually registers a local video and annotation file into the database."""
54             sport_str = args.sport.lower()
55             if sport_str not in self.config.get("supported_sports", ["badminton"]):
56                 print(f"Error: Sport '{sport_str}' is not supported. Supported:
{self.config.get('supported_sports')}")
57                 sys.exit(1)
58
59             # Validate video path
60             if not os.path.exists(args.video_path):
61                 print(f"Warning: Local video path '{args.video_path}' does not exist on
disk, but continuing registration.")
62
63             # Validate annotation file
64             if not os.path.exists(args.annotations_path):
65                 print(f"Error: Annotations file '{args.annotations_path}' not found.")
66                 sys.exit(1)
67
68             match_name = args.match_name or os.path.basename(args.video_path).split('.')[0]
69             video_id = args.video_id or match_name.lower().replace(" ", "_")
70
71             # Parse license
72             try:
73                 license_type = LicenseType(args.license)
74             except ValueError:

```

```

75         print(f"Warning: Unknown license '{args.license}'. Defaulting to Unknown
(non-commercial).")
76         license_type = LicenseType.UNKNOWN
77
78         is_commercial = self.validator.check_license_commercial(license_type)
79
80         # 1. Parse Annotations
81         annotations = []
82         try:
83             annotations = self._parse_local_annotations(sport_str, video_id,
args.annotations_path)
84         except Exception as e:
85             print(f"Error parsing annotations: {e}")
86             sys.exit(1)
87
88         # 2. Run Validation
89         is_valid = True
90         errors = []
91         if sport_str == "badminton":
92             is_valid, errors = self.validator.validate_badminton_rallies(annotations)
93         elif sport_str == "tennis":
94             is_valid, errors = self.validator.validate_tennis_points(annotations)
95         elif sport_str == "cricket":
96             is_valid, errors = self.validator.validate_cricket_deliveries(annotations)
97         elif sport_str == "pickleball":
98             is_valid, errors = self.validator.validate_pickleball_rallies(annotations)
99         elif sport_str == "tabletennis":
100             is_valid, errors = self.validator.validate_tabletennis_rallies(annotations)
101
102         if not is_valid:
103             print(f"Error: Annotations failed validation!")
104             for err in errors:
105                 print(f" - {err}")
106             sys.exit(1)
107
108         # 3. Create Video Metadata
109         video_meta = VideoMetadata(
110             video_id=video_id,
111             sport=SportType(sport_str),
112             video_url=None,
113             local_path=args.video_path,
114             license_type=license_type,
115             license_url=None,
116             is_commercial=is_commercial,
117             status=CurationStatus.CURATED, # Custom local imports bypass staging by
default
118             match_name=match_name,
119             fps=args.fps,
120             resolution=args.resolution
121         )
122
123         # 4. Insert into Database
124         self.db.insert_video(video_meta)
125
126         count = 0
127         if sport_str == "badminton":
128             count = self.db.insert_badminton_rallies(annotations)
129         elif sport_str == "tennis":
130             count = self.db.insert_tennis_points(annotations)
131         elif sport_str == "cricket":
132             count = self.db.insert_cricket_deliveries(annotations)
133         elif sport_str == "pickleball":
134             count = self.db.insert_pickleball_rallies(annotations)
135         elif sport_str == "tabletennis":
136             count = self.db.insert_tabletennis_rallies(annotations)

```

```

137
138         print(f"\nSuccessfully imported local video '{match_name}':")
139         print(f" - Video ID: {video_id}")
140         print(f" - Sport: {sport_str}")
141         print(f" - Commercial Use Allowed: {is_commercial} (License:
{license_type.value})")
142         print(f" - Registered {count} annotated segments.\n")
143
144     def _parse_local_annotations(self, sport: str, video_id: str, filepath: str) ->
List[Any]:
145         """Parses simple JSON or CSV annotations files."""
146         parsed_items = []
147         ext = os.path.splitext(filepath)[1].lower()
148
149         if ext == '.json':
150             with open(filepath, 'r', encoding='utf-8') as f:
151                 data = json.load(f)
152
153             if not isinstance(data, list):
154                 raise ValueError("JSON file must contain a list of objects.")
155
156             # Uses the same safe casts as the CSV path: a missing/blank required
157             # time raises a clear ValueError (caught by the caller) instead of a
158             # bare KeyError, and optional ints degrade to sensible defaults.
159             for idx, obj in enumerate(data):
160                 if sport == "badminton":
161                     parsed_items.append(BadmintonRally(
162                         video_id=video_id,
163                         rally_number=_safe_int(obj.get("rally_number"), idx + 1),
164                         start_time=_safe_float(obj.get("start_time")),
165                         end_time=_safe_float(obj.get("end_time")),
166                         score_before=obj.get("score_before"),
167                         score_after=obj.get("score_after"),
168                         shots_count=_safe_int(obj.get("shots_count"), None),
169                         ending_reason=obj.get("ending_reason"),
170                         last_shot_outcome=obj.get("last_shot_outcome")
171                     ))
172                 elif sport == "tennis":
173                     parsed_items.append(TennisPoint(
174                         video_id=video_id,
175                         point_number=_safe_int(obj.get("point_number"), idx + 1),
176                         start_time=_safe_float(obj.get("start_time")),
177                         end_time=_safe_float(obj.get("end_time")),
178                         set_score=obj.get("set_score"),
179                         game_score=obj.get("game_score"),
180                         server=obj.get("server"),
181                         ending_type=obj.get("ending_type")
182                     ))
183                 elif sport == "cricket":
184                     parsed_items.append(CricketDelivery(
185                         video_id=video_id,
186                         over=_safe_int(obj.get("over"), 0),
187                         ball=_safe_int(obj.get("ball"), 0),
188                         start_time=_safe_float(obj.get("start_time")),
189                         end_time=_safe_float(obj.get("end_time")),
190                         batsman=obj.get("batsman"),
191                         bowler=obj.get("bowler"),
192                         runs=_safe_int(obj.get("runs"), None),
193                         wicket=bool(obj.get("wicket", False)),
194                         extras=_safe_int(obj.get("extras"), 0)
195                     ))
196                 elif sport == "pickleball":
197                     parsed_items.append(PickleballRally(
198                         video_id=video_id,
199                         rally_number=_safe_int(obj.get("rally_number"), idx + 1),

```

```

200         start_time=_safe_float(obj.get("start_time")),
201         end_time=_safe_float(obj.get("end_time")),
202         score_before=obj.get("score_before"),
203         score_after=obj.get("score_after"),
204         shots_count=_safe_int(obj.get("shots_count"), None),
205         ending_reason=obj.get("ending_reason"),
206         last_shot_outcome=obj.get("last_shot_outcome")
207     ))
208 elif sport == "tabletennis":
209     parsed_items.append(TableTennisRally(
210         video_id=video_id,
211         rally_number=_safe_int(obj.get("rally_number"), idx + 1),
212         start_time=_safe_float(obj.get("start_time")),
213         end_time=_safe_float(obj.get("end_time")),
214         score_before=obj.get("score_before"),
215         score_after=obj.get("score_after"),
216         shots_count=_safe_int(obj.get("shots_count"), None),
217         ending_reason=obj.get("ending_reason"),
218         last_shot_outcome=obj.get("last_shot_outcome")
219     ))
220 elif ext == '.csv':
221     with open(filepath, 'r', encoding='utf-8') as f:
222         reader = csv.DictReader(f)
223         # Normalize headers
224         reader.fieldnames = [n.lower().strip() for n in reader.fieldnames] if
reader.fieldnames else []
225
226     for idx, row in enumerate(reader):
227         if sport == "badminton":
228             parsed_items.append(BadmintonRally(
229                 video_id=video_id,
230                 rally_number=_safe_int(row.get("rally_number"), idx + 1),
231                 start_time=_safe_float(row.get("start_time")),
232                 end_time=_safe_float(row.get("end_time")),
233                 score_before=row.get("score_before"),
234                 score_after=row.get("score_after"),
235                 shots_count=_safe_int(row.get("shots_count"), None),
236                 ending_reason=row.get("ending_reason"),
237                 last_shot_outcome=row.get("last_shot_outcome")
238             ))
239         elif sport == "tennis":
240             parsed_items.append(TennisPoint(
241                 video_id=video_id,
242                 point_number=_safe_int(row.get("point_number"), idx + 1),
243                 start_time=_safe_float(row.get("start_time")),
244                 end_time=_safe_float(row.get("end_time")),
245                 set_score=row.get("set_score"),
246                 game_score=row.get("game_score"),
247                 server=row.get("server"),
248                 ending_type=row.get("ending_type")
249             ))
250         elif sport == "cricket":
251             parsed_items.append(CricketDelivery(
252                 video_id=video_id,
253                 over=_safe_int(row.get("over"), 0),
254                 ball=_safe_int(row.get("ball"), 0),
255                 start_time=_safe_float(row.get("start_time")),
256                 end_time=_safe_float(row.get("end_time")),
257                 batsman=row.get("batsman"),
258                 bowler=row.get("bowler"),
259                 runs=_safe_int(row.get("runs"), None),
260                 wicket=(row.get("wicket") or "").lower() in ["true", "1",
"yes"],
261                 extras=_safe_int(row.get("extras"), 0)
262             ))

```

```

263         elif sport == "pickleball":
264             parsed_items.append(PickleballRally(
265                 video_id=video_id,
266                 rally_number=_safe_int(row.get("rally_number"), idx + 1),
267                 start_time=_safe_float(row.get("start_time")),
268                 end_time=_safe_float(row.get("end_time")),
269                 score_before=row.get("score_before"),
270                 score_after=row.get("score_after"),
271                 shots_count=_safe_int(row.get("shots_count"), None),
272                 ending_reason=row.get("ending_reason"),
273                 last_shot_outcome=row.get("last_shot_outcome")
274             ))
275         elif sport == "tabletennis":
276             parsed_items.append(TableTennisRally(
277                 video_id=video_id,
278                 rally_number=_safe_int(row.get("rally_number"), idx + 1),
279                 start_time=_safe_float(row.get("start_time")),
280                 end_time=_safe_float(row.get("end_time")),
281                 score_before=row.get("score_before"),
282                 score_after=row.get("score_after"),
283                 shots_count=_safe_int(row.get("shots_count"), None),
284                 ending_reason=row.get("ending_reason"),
285                 last_shot_outcome=row.get("last_shot_outcome")
286             ))
287     else:
288         raise ValueError("Unsupported annotations file extension. Use .json or
.csv.")
289
290     return parsed_items
291
292 def _fetch_one_with_retry(self, url, dest_path, args, max_height, max_size_mb):
293     """Download one video at best resolution, retrying transient failures.
294
295     A below-min-height result almost always means YouTube rate-limited the
296     request and served a low fallback (not that the video lacks HD), so we
297     retry with a growing backoff. Returns (DownloadResult, None) on success,
298     or (None, failure_reason) once retries are exhausted. A below-min file is
299     discarded each attempt so it never lands in the DB.
300     """
301     attempts = max(1, args.retries + 1)
302     reason = "unknown error"
303     for i in range(attempts):
304         result = download_best_video(
305             url, dest_path,
306             max_height=max_height, max_size_mb=max_size_mb,
307             cookies_from_browser=args.cookies_from_browser,
308             cookies_file=args.cookies,
309             player_client=args.player_client,
310         )
311         if result.success and (result.height is None or result.height >=
args.min_height):
312             return result, None
313
314         if not result.success:
315             reason = result.error or "unknown error"
316         else:
317             reason = (f"got {result.resolution_label} (< min-height
{args.min_height}p) ? "
318                     f"YouTube served a low format (rate-limited?)")
319         try:
320             os.remove(result.path)
321         except OSError:
322             pass
323
324     if i < attempts - 1:

```

```

325         backoff = args.retry_backoff * (i + 1)
326         print(f" attempt {i+1}/{attempts} failed: {reason}\n retrying in
{backoff:.0f}s...")
327         time.sleep(backoff)
328     return None, reason
329
330 def fetch(self, args):
331     """(Re)download videos at best available resolution, updating the DB in place.
332
333     This is the in-place upgrade path: rows, rally annotations, and curation
334     status are all preserved ? only the video file plus its `local_path`,
335     `resolution`, and `fps` are refreshed. By default it skips videos that
336     already have a local file at or above `--min-height`, so re-runs are cheap
337     and resumable; `--force` re-fetches everything.
338     """
339     storage = self.config.get("video_storage", {})
340     local_dir = storage.get("local_dir", "./data/videos")
341     max_download_size_mb = storage.get("max_download_size_mb")
342     max_height = args.max_height if args.max_height is not None else
storage.get("max_height")
343     os.makedirs(local_dir, exist_ok=True)
344
345     commercial_only = not args.include_noncommercial
346     if args.status == "all":
347         statuses = [CurationStatus.CURATED, CurationStatus.STAGING]
348     else:
349         statuses = [CurationStatus(args.status)]
350
351     supported = self.config.get("supported_sports", ["badminton"])
352     sport_strs = [args.sport] if args.sport else supported
353
354     # Collect target videos.
355     videos = []
356     for sport_str in sport_strs:
357         try:
358             sport = SportType(sport_str.lower())
359         except ValueError:
360             print(f"Warning: skipping unknown sport '{sport_str}'.")
361             continue
362         for status in statuses:
363             videos.extend(self.db.get_videos_by_status(status, sport))
364     if args.video_id:
365         videos = [v for v in videos if v.video_id == args.video_id]
366
367     if not videos:
368         print("\nNo matching videos to fetch.\n")
369         return
370
371     results = [] # (video_id, action, detail)
372     for video in videos:
373         if commercial_only and not video.is_commercial:
374             results.append((video.video_id, "skip", "non-commercial"))
375             continue
376         if not video.video_url or "watch?v=example" in video.video_url:
377             # Either no URL, or the parser's placeholder (the ShuttleSet catalog
378             # had no URL for this match) ? nothing to download.
379             results.append((video.video_id, "skip", "no real source URL"))
380             continue
381
382         dest_path = os.path.join(local_dir, f"{video.video_id}.mp4")
383
384         # Skip if we already have a good-enough local file (unless --force).
385         if not args.force and os.path.exists(dest_path):
386             _, height, _ = probe_resolution_fps(dest_path)
387             if height is not None and height >= args.min_height:

```



```

388             results.append((video.video_id, "have", f"{height_to_label(height)}
already"))
389             continue
390
391         print(f"Fetching {video.video_id} ({video.match_name})...")
392         result, fail_reason = self._fetch_one_with_retry(
393             video.video_url, dest_path, args, max_height, max_download_size_mb)
394         if fail_reason is not None:
395             results.append((video.video_id, "FAIL", fail_reason))
396             if args.sleep:
397                 time.sleep(args.sleep)
398             continue
399
400         # Update only the file-derived fields; preserve everything else (UPSERT
401         # mutates the row in place, leaving rallies intact).
402         video.local_path = result.path
403         if result.resolution_label:
404             video.resolution = result.resolution_label
405         if result.fps:
406             video.fps = result.fps
407         self.db.insert_video(video)
408
409         detail = (f"{result.resolution_label or '?'}"
410                 f"{f' {result.fps:g}fps' if result.fps else ''},
{result.size_mb:.0f} MB")
411         results.append((video.video_id, "fetched", detail))
412         if args.sleep:
413             time.sleep(args.sleep)
414
415         print("\n=== Fetch summary ===")
416         print(tabulate([[vid, action, detail] for vid, action, detail in results],
417                       headers=["Video ID", "Action", "Detail"], tablefmt="grid"))
418         fetched = sum(1 for _, a, _ in results if a == "fetched")
419         failed = [r for r in results if r[1] == "FAIL"]
420         print(f"\nFetched/updated : {fetched}    Already-good : {sum(1 for _,a,_ in
results if a=='have'))}    "
421               f"Skipped : {sum(1 for _,a,_ in results if a=='skip')}}    Failed :
{len(failed)}}")
422         if failed:
423             print("Failures (left unchanged in DB):")
424             for vid, _, detail in failed:
425                 print(f"    - {vid}: {detail}")
426         print()
427
428     def export_eval_set(self, args):
429         """Export curated annotations as indexer-ready eval/training sets.
430
431         Emits, per qualifying video, a `

```

```

449
450 manifest = []
451 skipped_noncomm = 0
452 skipped_empty = 0
453
454 for sport_str in sport_strs:
455     try:
456         sport = SportType(sport_str.lower())
457     except ValueError:
458         print(f"Warning: skipping unknown sport '{sport_str}'.")
459         continue
460
461     videos = []
462     for status in statuses:
463         videos.extend(self.db.get_videos_by_status(status, sport))
464
465     for video in videos:
466         if commercial_only and not video.is_commercial:
467             skipped_noncomm += 1
468             continue
469
470         intervals = self.db.get_segment_intervals(video.video_id, sport)
471         if not intervals:
472             skipped_empty += 1
473             continue
474
475         sport_dir = os.path.join(out_dir, sport.value)
476         os.makedirs(sport_dir, exist_ok=True)
477         csv_path = os.path.join(sport_dir, f"{video.video_id}.csv")
478         with open(csv_path, "w", newline="", encoding="utf-8") as f:
479             writer = csv.writer(f)
480             writer.writerow(["start", "end"])
481             for start, end in intervals:
482                 writer.writerow([start, end])
483
484         manifest.append({
485             "video_id": video.video_id,
486             "sport": sport.value,
487             "csv": os.path.relpath(csv_path, out_dir).replace(os.sep, "/"),
488             "local_path": video.local_path,
489             "video_url": video.video_url,
490             "match_name": video.match_name,
491             "fps": video.fps,
492             "resolution": video.resolution,
493             "license_type": video.license_type.value,
494             "is_commercial": video.is_commercial,
495             "status": video.status.value,
496             "num_segments": len(intervals),
497         })
498
499     os.makedirs(out_dir, exist_ok=True)
500     manifest_doc = {
501         "commercial_only": commercial_only,
502         "statuses": [s.value for s in statuses],
503         "total_videos": len(manifest),
504         "total_segments": sum(m["num_segments"] for m in manifest),
505         "eval_sets": manifest,
506     }
507     manifest_path = os.path.join(out_dir, "manifest.json")
508     with open(manifest_path, "w", encoding="utf-8") as f:
509         json.dump(manifest_doc, f, indent=2)
510
511     print(f"\n=== Exported eval/training set to '{out_dir}' ===")
512     if manifest:
513         summary = [[m["sport"], m["video_id"], m["num_segments"],

```

```

514                 "YES" if m["is_commercial"] else "NO",
515                 os.path.basename(m["local_path"]) if m["local_path"] else "(no
local file)"]
516             for m in manifest]
517         print(tabulate(summary, headers=["Sport", "Video ID", "Segments", "Comm?",
"Video file"], tablefmt="grid"))
518         print(f"\nVideos exported : {len(manifest)}")
519         print(f"Total segments : {manifest_doc['total_segments']}")
520         if skipped_noncomm:
521             print(f"Skipped (non-commercial) : {skipped_noncomm} (pass --include-
noncommercial to include)")
522         if skipped_empty:
523             print(f"Skipped (no annotations) : {skipped_empty}")
524         print(f"Manifest : {manifest_path}\n")
525
526     def convert_cvat(self, args):
527         """Convert a vendor CVAT rally export into our canonical per-rally CSV.
528
529         Absorbs the CVAT bounding-box delivery format (rally fields carried as box
530         attributes) on the ingestion side: each rally's time is recomputed from
531         its frame range at the video's true fps, and content defects (overlaps,
532         zero-duration, off-vocabulary endings, coarse sampling) are written to an
533         issues report rather than silently "fixed". The emitted CSV imports via
534         `import-local`; any blocker issue must be resolved first, since the
535         importer's validator rejects overlapping / zero-duration rallies.
536         """
537         from src.parsers.cvat.rally_cvat import (
538             convert, write_canonical_csv, render_issue_report,
539         )
540         try:
541             result = convert(args.input, fps=args.fps,
542                             video_path=args.video_path, step=args.step)
543         except (FileNotFoundError, ValueError) as e:
544             print(f"Error: {e}")
545             sys.exit(1)
546
547         write_canonical_csv(result.records, args.out)
548         report = render_issue_report(result, os.path.basename(args.input))
549         if args.issues:
550             with open(args.issues, "w", encoding="utf-8") as f:
551                 f.write(report)
552         print(report)
553         print(f"Canonical CSV written: {args.out} ({len(result.records)} rallies)")
554         if result.n_blockers:
555             print(f"\n {result.n_blockers} BLOCKER issue(s) ? import-local will reject
"
556                 f"this file until they are resolved (by the vendor or manual
review).")
557         else:
558             print("\n No blockers. Next:")
559             print(f"    python -m src.cli.curate import-local --sport <sport> "
560                   f"--video-path <video> --annotations-path {args.out} --fps
{result.fps:g}")
561
562     def validate_delivery(self, args):
563         """Run the full validation suite on a rally delivery and emit a report.
564
565         Accepts a CVAT export (--input + --video-path/--fps) or an already-canonical
566         CSV (--csv). Writes, into --out-dir:
567         <id>_report.md    human report (scoreboard + issues + rallies to review)
568         <id>_report.json  machine summary (for CI / a bulk-ingest loop)
569         <id>_review.csv   the annotatable manual-verification sheet
570         Exits non-zero if any blocker issue exists, so it can gate a bulk pipeline.
571         """
572         import json

```

```

573     from src.parsers.cvat.rally_cvat import (
574         convert, load_canonical_csv, detect_issues,
575         build_summary, render_validation_md, write_review_csv,
576     )
577     vid = args.video_id or "delivery"
578     fps = None
579     if args.input:
580         try:
581             result = convert(args.input, fps=args.fps, video_path=args.video_path)
582         except (FileNotFoundError, ValueError) as e:
583             print(f"Error: {e}")
584             sys.exit(1)
585         records, issues, fps = result.records, result.issues, result.fps
586     elif args.csv:
587         try:
588             records = load_canonical_csv(args.csv)
589         except FileNotFoundError as e:
590             print(f"Error: {e}")
591             sys.exit(1)
592         if not records:
593             print(f"Error: no usable rally rows in {args.csv}")
594             sys.exit(1)
595         issues = detect_issues(records)
596     else:
597         print("Error: provide --input <cvat.xml> or --csv <canonical.csv>")
598         sys.exit(1)
599
600     os.makedirs(args.out_dir, exist_ok=True)
601     summary = build_summary(records, issues, source=vid, fps=fps)
602     md = render_validation_md(records, issues, source=vid)
603     json_path = os.path.join(args.out_dir, f"{vid}_report.json")
604     md_path = os.path.join(args.out_dir, f"{vid}_report.md")
605     review_path = os.path.join(args.out_dir, f"{vid}_review.csv")
606     with open(json_path, "w", encoding="utf-8") as f:
607         json.dump(summary, f, indent=2)
608     with open(md_path, "w", encoding="utf-8") as f:
609         f.write(md)
610     write_review_csv(records, issues, review_path)
611
612     print(md)
613     print(f"\nWrote:\n {md_path}\n {json_path}\n {review_path}")
614     if summary["n_blockers"]:
615         print(f"\n GATE: BLOCKED ? {summary['n_blockers']} blocker(s) must be
resolved.")
616         sys.exit(2)
617     print("\n GATE: PASS ? review the flagged rallies, then `import-local`.")
618
619     def curate(self):
620         """Interactive loop to review and curate staging video annotations."""
621         staging_videos = self.db.get_videos_by_status(CurationStatus.STAGING)
622         if not staging_videos:
623             print("\nNo staging video annotations to review.\n")
624             return
625
626         print(f"\nFound {len(staging_videos)} matches in STAGING state for curation:")
627         table_data = []
628         for idx, video in enumerate(staging_videos):
629             table_data.append([
630                 idx + 1,
631                 video.video_id,
632                 video.match_name,
633                 video.sport.value,
634                 video.license_type.value,
635                 "YES" if video.is_commercial else "NO"
636             ])

```

```

637
638         print(tabulate(table_data, headers=["#", "Video ID", "Match Name", "Sport",
"License", "Commercial Safe?"], tablefmt="grid"))
639         print("\nEnter a number (1-{}) to review details, or 'q' to
quit:".format(len(staging_videos)))
640
641         choice = input("> ").strip()
642         if choice.lower() == 'q':
643             return
644
645         try:
646             idx = int(choice) - 1
647             if idx < 0 or idx >= len(staging_videos):
648                 print("Invalid choice.")
649                 return
650             self._curate_video_interactive(staging_videos[idx])
651         except ValueError:
652             print("Invalid input.")
653
654     def _curate_video_interactive(self, video: VideoMetadata):
655         """Displays details of a specific staging video and prompts for curation
action."""
656
657         # Load and count events
658         rallies = []
659         pts = []
660         delivs = []
661         if video.sport == SportType.BADMINTON:
662             rallies = self.db.get_badminton_rallies(video.video_id)
663         elif video.sport == SportType.PICKLEBALL:
664             rallies = self.db.get_pickleball_rallies(video.video_id)
665         elif video.sport == SportType.TABLE_TENNIS:
666             rallies = self.db.get_tabletennis_rallies(video.video_id)
667         elif video.sport == SportType.TENNIS:
668             pts = self.db.get_tennis_points(video.video_id)
669         elif video.sport == SportType.CRICKET:
670             delivs = self.db.get_cricket_deliveries(video.video_id)
671
672         while True:
673             print("\n===== REVIEWING VIDEO =====")
674             print(f"Match Name : {video.match_name}")
675             print(f"Video ID : {video.video_id}")
676             print(f"Sport : {video.sport.value}")
677             print(f"Video URL : {video.video_url or 'N/A'}")
678             print(f"Local Path : {video.local_path or 'N/A'}")
679             print(f"License : {video.license_type.value} ({video.license_url or 'no
URL'})")
680             print(f"Commercial OK: {video.is_commercial}")
681
682             if video.sport in (SportType.BADMINTON, SportType.PICKLEBALL,
SportType.TABLE_TENNIS):
683                 print(f"Total Rallies: {len(rallies)}")
684                 if rallies:
685                     print("Sample rallies:")
686                     sample_data = [[r.rally_number, f"{r.start_time}s - {r.end_time}s",
r.score_after, r.shots_count] for r in rallies[:5]]
687                     print(tabulate(sample_data, headers=["Rally #", "Duration", "Score",
"Shots"], tablefmt="simple"))
688                 elif video.sport == SportType.TENNIS:
689                     print(f"Total Points : {len(pts)}")
690                     if pts:
691                         print("Sample points:")
692                         sample_data = [[p.point_number, f"{p.start_time}s - {p.end_time}s",
p.game_score] for p in pts[:5]]
693                         print(tabulate(sample_data, headers=["Point #", "Duration",

```

```

"Score"], tablefmt="simple"))
694         elif video.sport == SportType.CRICKET:
695             print(f"Total Balls : {len(delivs)}")
696             if delivs:
697                 print("Sample deliveries:")
698                 sample_data = [[f"{d.over}.{d.ball}", f"{d.start_time}s -
{d.end_time}s", d.runs, "Wicket" if d.wicket else ""] for d in delivs[:5]]
699                 print(tabulate(sample_data, headers=["Over.Ball", "Duration",
"Runs", "Wicket"], tablefmt="simple"))
700
701             print("\nChoose Curation Action:")
702             print(" [a] Approve (promote to CURATED)")
703             print(" [r] Reject (delete from database)")
704             print(" [p] Play a rally/point segment in browser")
705             print(" [k] Keep in Staging (do nothing and go back)")
706
707             action = input("> ").strip().lower()
708             if action == 'a':
709                 self.db.update_status(video.video_id, CurationStatus.CURATED)
710                 print(f"Successfully promoted '{video.match_name}' to CURATED.\n")
711                 break
712             elif action == 'r':
713                 self.db.delete_video(video.video_id)
714                 print(f"Successfully rejected and deleted '{video.match_name}'.\n")
715                 break
716             elif action == 'k':
717                 print("Curation state unchanged.\n")
718                 break
719             elif action == 'p':
720                 self._play_segment_interactive(video, rallies, pts, delivs)
721             else:
722                 print("Invalid action selected.")
723
724         def _play_segment_interactive(self, video: VideoMetadata, rallies:
List[BadmintonRally], pts: List[TennisPoint], delivs: List[CricketDelivery]):
725             import webbrowser
726
727             # Get start/end based on user input
728             start_time = None
729             end_time = None
730             label = "segment"
731
732             if video.sport in (SportType.BADMINTON, SportType.PICKLEBALL,
SportType.TABLE_TENNIS):
733                 if not rallies:
734                     print("No rallies to play.")
735                     return
736                 idx_str = input(f"Enter Rally number to play (1-{len(rallies)}): ").strip()
737                 try:
738                     rally_num = int(idx_str)
739                     rally = next((r for r in rallies if r.rally_number == rally_num), None)
740                     if rally:
741                         start_time = rally.start_time
742                         end_time = rally.end_time
743                         label = f"Rally {rally_num}"
744                 except ValueError:
745                     pass
746             elif video.sport == SportType.TENNIS:
747                 if not pts:
748                     print("No points to play.")
749                     return
750                 idx_str = input(f"Enter Point number to play (1-{len(pts)}): ").strip()
751                 try:
752                     pt_num = int(idx_str)
753                     pt = next((p for p in pts if p.point_number == pt_num), None)

```

```

754         if pt:
755             start_time = pt.start_time
756             end_time = pt.end_time
757             label = f"Point {pt_num}"
758     except ValueError:
759         pass
760 elif video.sport == SportType.CRICKET:
761     if not delivs:
762         print("No deliveries to play.")
763         return
764     over_str = input("Enter Over number: ").strip()
765     ball_str = input("Enter Ball number: ").strip()
766     try:
767         o_num = int(over_str)
768         b_num = int(ball_str)
769         d = next((x for x in delivs if x.over == o_num and x.ball == b_num),
None)
770         if d:
771             start_time = d.start_time
772             end_time = d.end_time
773             label = f"Over {o_num} Ball {b_num}"
774     except ValueError:
775         pass
776
777     if start_time is None or end_time is None:
778         print("Segment not found or invalid input.")
779         return
780
781     print(f"Playing {label} (start: {start_time}s, end: {end_time}s)...")
782
783     # Check video location
784     play_url = None
785     if video.local_path and os.path.exists(video.local_path):
786         abs_path = os.path.abspath(video.local_path)
787         # Create file:// URL with media fragment (browsers support #t=start,end
fragment)
788         play_url = f"file:/// {abs_path.replace(os.sep,
'/' )}#t={start_time},{end_time}"
789     elif video.video_url:
790         # YouTube URL: seek to start time
791         if "youtube.com" in video.video_url or "youtu.be" in video.video_url:
792             t_sec = int(start_time)
793             sep = "&" if "?" in video.video_url else "?"
794             play_url = f"{video.video_url}{sep}t={t_sec}s"
795         else:
796             play_url = video.video_url
797
798     if play_url:
799         print(f"Opening browser: {play_url}")
800         webbrowser.open(play_url)
801     else:
802         print("Error: No local video file or remote URL available to play.")
803
804
805 def main():
806     parser = argparse.ArgumentParser(description="Sports Data Curator CLI")
807     subparsers = parser.add_subparsers(dest="command")
808
809     # Status command
810     subparsers.add_parser("status", help="Display summary status of sports metadata
database")
811
812     # Curate command
813     subparsers.add_parser("curate", help="Launch interactive curation tool for staging
entries")

```

```

814
815     # Import-local command
816     import_parser = subparsers.add_parser("import-local", help="Manually import a local
video and annotations")
817     import_parser.add_argument("--sport", required=True, help="Sport type (e.g.
badminton, tennis, cricket)")
818     import_parser.add_argument("--video-path", required=True, help="Path to local video
file")
819     import_parser.add_argument("--annotations-path", required=True, help="Path to local
JSON/CSV annotations file")
820     import_parser.add_argument("--match-name", help="Name of the match (defaults to
video filename)")
821     import_parser.add_argument("--video-id", help="Custom alphanumeric video ID")
822     import_parser.add_argument("--license", default="Proprietary", help="License type
(e.g. MIT, CC-BY-NC, Proprietary)")
823     import_parser.add_argument("--fps", type=float, default=30.0, help="Frames per
second of the video")
824     import_parser.add_argument("--resolution", default="1080p", help="Resolution of the
video (e.g., 1080p, 720p)")
825
826     # Fetch command ? (re)download videos at best resolution, update DB in place
827     fetch_parser = subparsers.add_parser(
828         "fetch", help="(Re)download videos at best available resolution, updating the DB
in place")
829     fetch_parser.add_argument("--sport", help="Restrict to a single sport (default: all
supported_sports)")
830     fetch_parser.add_argument("--video-id", help="Fetch a single video by ID")
831     fetch_parser.add_argument("--status", choices=["staging", "curated", "all"],
default="all",
832                                     help="Which curation statuses to fetch (default: all)")
833     fetch_parser.add_argument("--min-height", type=int, default=720,
834                                     help="Skip videos already at/above this height unless
--force (default: 720)")
835     fetch_parser.add_argument("--max-height", type=int, default=None,
836                                     help="Cap download resolution (e.g. 1080); default: best
available")
837     fetch_parser.add_argument("--force", action="store_true",
838                                     help="Re-download even if a good-enough local file already
exists")
839     fetch_parser.add_argument("--include-noncommercial", action="store_true",
840                                     help="Include videos whose license is not commercially
safe (default: commercial only)")
841     fetch_parser.add_argument("--cookies-from-browser", dest="cookies_from_browser",
metavar="BROWSER",
842                                     help="Use a logged-in browser's YouTube cookies (e.g.
chrome, edge, firefox, brave). "
843                                     "A Premium login gives reliable best-resolution
downloads. "
844                                     "Chromium browsers must be CLOSED on Windows (cookie
DB lock).")
845     fetch_parser.add_argument("--cookies", metavar="FILE",
846                                     help="Path to a Netscape-format cookies.txt (alternative
to --cookies-from-browser)")
847     fetch_parser.add_argument("--player-client", dest="player_client",
default="default", metavar="CLIENT",
848                                     help="YouTube player client yt-dlp uses (default:
'default'). Avoid web/web_safari "
849                                     "(SABR-forced -> drops to 144p). 'tv'/'mweb' also
serve 1080p if needed.)")
850     fetch_parser.add_argument("--sleep", type=float, default=4.0, metavar="SECONDS",
851                                     help="Pause between videos to avoid YouTube rate-limiting
(default: 4)")
852     fetch_parser.add_argument("--retries", type=int, default=2, metavar="N",
853                                     help="Retries per video when a download fails or returns a
low format (default: 2)")

```



```

854         fetch_parser.add_argument("--retry-backoff", type=float, default=20.0,
metavar="SECONDS",
855                                     help="Base backoff between retries; grows linearly per
attempt (default: 20)")
856
857         # Convert-cvat command ? normalize a vendor CVAT export into our canonical CSV
858         cvat_parser = subparsers.add_parser(
859             "convert-cvat",
860             help="Convert a vendor CVAT rally export (bounding-box XML) into the canonical
per-rally CSV")
861         cvat_parser.add_argument("--input", required=True,
862                                 help="Path to the CVAT XML export (image- or track-mode)")
863         cvat_parser.add_argument("--out", required=True,
864                                 help="Path to write the canonical per-rally CSV (import-
local input)")
865         cvat_parser.add_argument("--video-path",
866                                 help="Source video; its fps is read via ffprobe (seconds =
frame / fps)")
867         cvat_parser.add_argument("--fps", type=float,
868                                 help="Override fps instead of probing the video")
869         cvat_parser.add_argument("--step", type=int,
870                                 help="Frame sampling step (default: read from CVAT meta,
else inferred)")
871         cvat_parser.add_argument("--issues",
872                                 help="Path to also write the human-readable issues report")
873
874         # Validate-delivery command ? run the full check suite + emit a structured report
875         vd_parser = subparsers.add_parser(
876             "validate-delivery",
877             help="Validate a rally delivery (CVAT export or canonical CSV) and emit report +
review sheet")
878         vd_parser.add_argument("--input", help="CVAT XML export (use with --video-
path/--fps)")
879         vd_parser.add_argument("--csv", help="An already-canonical per-rally CSV to
validate")
880         vd_parser.add_argument("--video-path",
881                                 help="Source video; fps via ffprobe (when using --input)")
882         vd_parser.add_argument("--fps", type=float, help="Override fps (when using
--input)")
883         vd_parser.add_argument("--video-id", help="Names the report files (<id>_report.md,
etc.)")
884         vd_parser.add_argument("--out-dir", default="data/validation",
885                                 help="Where to write the report + review sheet")
886
887         # Export command
888         export_parser = subparsers.add_parser(
889             "export", help="Export curated annotations as indexer-ready eval/training CSVs +
manifest")
890         export_parser.add_argument("--out-dir", default="data/eval_export",
891                                 help="Output directory for per-video CSVs and
manifest.json")
892         export_parser.add_argument("--sport",
893                                 help="Restrict to a single sport (default: all
supported_sports)")
894         export_parser.add_argument("--include-staging", action="store_true",
895                                 help="Also export videos still in staging (default:
curated only)")
896         export_parser.add_argument("--include-noncommercial", action="store_true",
897                                 help="Include videos whose license is not commercially
safe (default: commercial only)")
898
899         args = parser.parse_args()
900         cli = SportsDataCLI()
901
902         if args.command == "status":

```

```

903         cli.status()
904     elif args.command == "curate":
905         cli.curate()
906     elif args.command == "import-local":
907         cli.import_local(args)
908     elif args.command == "convert-cvat":
909         cli.convert_cvat(args)
910     elif args.command == "validate-delivery":
911         cli.validate_delivery(args)
912     elif args.command == "export":
913         cli.export_eval_set(args)
914     elif args.command == "fetch":
915         cli.fetch(args)
916     else:
917         # Default: print help
918         parser.print_help()
919
920 if __name__ == "__main__":
921     main()
922

```

8. assistant (2026-06-07T06:41:48.191Z)

Now let me explore the database schema and the ingest CLI:

9. user (2026-06-07T06:41:48.498Z)

```

1     import sqlite3
2     import os
3     from contextlib import contextmanager
4     from typing import List, Dict, Any, Optional
5     from src.core.schemas.base import VideoMetadata, SportType, CurationStatus, LicenseType
6     from src.core.schemas.badminton import BadmintonRally
7     from src.core.schemas.tennis import TennisPoint
8     from src.core.schemas.cricket import CricketDelivery
9     from src.core.schemas.pickleball import PickleballRally
10    from src.core.schemas.tabletennis import TableTennisRally
11
12    # Maps each sport to its annotation table and the column(s) that order segments
13    # chronologically. All child tables expose (start_time, end_time), so a
14    # single generic reader can serve every sport ? adding a new sport is one entry
15    # here plus its CREATE TABLE, with no new query code.
16    SEGMENT_TABLES = {
17        SportType.BADMINTON: ("badminton_rallies", "rally_number"),
18        SportType.TENNIS: ("tennis_points", "point_number"),
19        SportType.CRICKET: ("cricket_deliveries", "over_num, ball_num"),
20        SportType.PICKLEBALL: ("pickleball_rallies", "rally_number"),
21        SportType.TABLE_TENNIS: ("tabletennis_rallies", "rally_number"),
22    }
23
24
25    class SportsDatabase:
26        def __init__(self, db_path: str):
27            self.db_path = db_path
28            # Ensure target directory exists
29            os.makedirs(os.path.dirname(os.path.abspath(db_path)), exist_ok=True)
30            self._init_db()
31
32        @contextmanager
33        def _get_connection(self):
34            """Yield a SQLite connection and always close it.
35
36            Used as ``with self._get_connection() as conn:``. A plain
37            ``sqlite3.connect(...)`` used as a context manager commits/rolls back the
38            transaction but does NOT close the connection, which leaks handles and

```

```

39         raises ResourceWarnings. This wrapper guarantees the connection is closed
40         on exit; writers still commit explicitly inside the block.
41         """
42         conn = sqlite3.connect(self.db_path)
43         conn.row_factory = sqlite3.Row
44         # SQLite disables foreign keys per-connection by default; enable so the
45         # ON DELETE CASCADE declarations on the child tables actually fire.
46         conn.execute("PRAGMA foreign_keys = ON")
47         try:
48             yield conn
49         finally:
50             conn.close()
51
52     def _init_db(self):
53         with self._get_connection() as conn:
54             cursor = conn.cursor()
55
56             # 1. Create common videos table
57             cursor.execute("""
58             CREATE TABLE IF NOT EXISTS videos (
59                 video_id TEXT PRIMARY KEY,
60                 sport TEXT NOT NULL,
61                 video_url TEXT,
62                 local_path TEXT,
63                 license_type TEXT NOT NULL,
64                 license_url TEXT,
65                 is_commercial INTEGER NOT NULL DEFAULT 0,
66                 status TEXT NOT NULL DEFAULT 'staging',
67                 match_name TEXT NOT NULL,
68                 fps REAL,
69                 resolution TEXT
70             )
71             """)
72
73             # 2. Create badminton_rallies table
74             cursor.execute("""
75             CREATE TABLE IF NOT EXISTS badminton_rallies (
76                 video_id TEXT NOT NULL,
77                 rally_number INTEGER NOT NULL,
78                 start_time REAL NOT NULL,
79                 end_time REAL NOT NULL,
80                 score_before TEXT,
81                 score_after TEXT,
82                 shots_count INTEGER,
83                 ending_reason TEXT,
84                 last_shot_outcome TEXT,
85                 PRIMARY KEY (video_id, rally_number),
86                 FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
87             )
88             """)
89
90             # 3. Create tennis_points table
91             cursor.execute("""
92             CREATE TABLE IF NOT EXISTS tennis_points (
93                 video_id TEXT NOT NULL,
94                 point_number INTEGER NOT NULL,
95                 start_time REAL NOT NULL,
96                 end_time REAL NOT NULL,
97                 set_score TEXT,
98                 game_score TEXT,
99                 server TEXT,
100                 ending_type TEXT,
101                 PRIMARY KEY (video_id, point_number),
102                 FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
103             )

```

```

104         """)
105
106     # 4. Create cricket_deliveries table
107     cursor.execute("""
108     CREATE TABLE IF NOT EXISTS cricket_deliveries (
109         video_id TEXT NOT NULL,
110         over_num INTEGER NOT NULL,
111         ball_num INTEGER NOT NULL,
112         start_time REAL NOT NULL,
113         end_time REAL NOT NULL,
114         batsman TEXT,
115         bowler TEXT,
116         runs INTEGER,
117         wicket INTEGER DEFAULT 0,
118         extras INTEGER DEFAULT 0,
119         PRIMARY KEY (video_id, over_num, ball_num),
120         FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
121     )
122     """)
123
124     # 5. Create pickleball_rallies table
125     cursor.execute("""
126     CREATE TABLE IF NOT EXISTS pickleball_rallies (
127         video_id TEXT NOT NULL,
128         rally_number INTEGER NOT NULL,
129         start_time REAL NOT NULL,
130         end_time REAL NOT NULL,
131         score_before TEXT,
132         score_after TEXT,
133         shots_count INTEGER,
134         ending_reason TEXT,
135         last_shot_outcome TEXT,
136         PRIMARY KEY (video_id, rally_number),
137         FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
138     )
139     """)
140
141     # 6. Create tabletennis_rallies table
142     cursor.execute("""
143     CREATE TABLE IF NOT EXISTS tabletennis_rallies (
144         video_id TEXT NOT NULL,
145         rally_number INTEGER NOT NULL,
146         start_time REAL NOT NULL,
147         end_time REAL NOT NULL,
148         score_before TEXT,
149         score_after TEXT,
150         shots_count INTEGER,
151         ending_reason TEXT,
152         last_shot_outcome TEXT,
153         PRIMARY KEY (video_id, rally_number),
154         FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
155     )
156     """)
157
158     # Migration: add last_shot_outcome to rally tables that predate the
159     # column. CREATE TABLE IF NOT EXISTS won't alter an existing table, so
160     # do it explicitly and idempotently (skip if the column is already there).
161     for tbl in ("badminton_rallies", "pickleball_rallies",
162 "tabletennis_rallies"):
163         existing = [r[1] for r in cursor.execute(f"PRAGMA
164 table_info({tbl})").fetchall()]
165         if "last_shot_outcome" not in existing:
166             cursor.execute(f"ALTER TABLE {tbl} ADD COLUMN last_shot_outcome
167 TEXT")

```

```

166         conn.commit()
167
168     def insert_video(self, video: VideoMetadata) -> bool:
169         """Insert a video entry, or update it in place if it already exists.
170
171         Uses an UPSERT rather than INSERT OR REPLACE: REPLACE would DELETE the
172         existing row, which (with foreign keys enabled) would cascade-delete the
173         video's child annotations. UPSERT mutates the row in place and leaves
174         children untouched.
175         """
176         with self._get_connection() as conn:
177             cursor = conn.cursor()
178             cursor.execute("""
179             INSERT INTO videos (
180                 video_id, sport, video_url, local_path, license_type,
181                 license_url, is_commercial, status, match_name, fps, resolution
182             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
183             ON CONFLICT(video_id) DO UPDATE SET
184                 sport=excluded.sport,
185                 video_url=excluded.video_url,
186                 local_path=excluded.local_path,
187                 license_type=excluded.license_type,
188                 license_url=excluded.license_url,
189                 is_commercial=excluded.is_commercial,
190                 status=excluded.status,
191                 match_name=excluded.match_name,
192                 fps=excluded.fps,
193                 resolution=excluded.resolution
194             """, (
195                 video.video_id,
196                 video.sport.value,
197                 video.video_url,
198                 video.local_path,
199                 video.license_type.value,
200                 video.license_url,
201                 1 if video.is_commercial else 0,
202                 video.status.value,
203                 video.match_name,
204                 video.fps,
205                 video.resolution
206             ))
207             conn.commit()
208             return True
209
210     def insert_badminton_rallies(self, rallies: List[BadmintonRally]) -> int:
211         if not rallies:
212             return 0
213         with self._get_connection() as conn:
214             cursor = conn.cursor()
215             # First clear existing rallies for this video_id to avoid key conflicts on
216             updates
217             video_id = rallies[0].video_id
218             cursor.execute("DELETE FROM badminton_rallies WHERE video_id = ?",
219             (video_id,))
220
221             for rally in rallies:
222                 cursor.execute("""
223                 INSERT INTO badminton_rallies (
224                     video_id, rally_number, start_time, end_time,
225                     score_before, score_after, shots_count, ending_reason,
226                     last_shot_outcome
227                 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
228                 """, (
229                     rally.video_id,
230                     rally.rally_number,

```

```

228         rally.start_time,
229         rally.end_time,
230         rally.score_before,
231         rally.score_after,
232         rally.shots_count,
233         rally.ending_reason,
234         rally.last_shot_outcome
235     ))
236     conn.commit()
237     return len(rallies)
238
239 def insert_tennis_points(self, points: List[TennisPoint]) -> int:
240     if not points:
241         return 0
242     with self._get_connection() as conn:
243         cursor = conn.cursor()
244         video_id = points[0].video_id
245         cursor.execute("DELETE FROM tennis_points WHERE video_id = ?", (video_id,))
246
247         for pt in points:
248             cursor.execute("""
249             INSERT INTO tennis_points (
250                 video_id, point_number, start_time, end_time,
251                 set_score, game_score, server, ending_type
252             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
253             """, (
254                 pt.video_id,
255                 pt.point_number,
256                 pt.start_time,
257                 pt.end_time,
258                 pt.set_score,
259                 pt.game_score,
260                 pt.server,
261                 pt.ending_type
262             ))
263         conn.commit()
264         return len(points)
265
266 def insert_cricket_deliveries(self, deliveries: List[CricketDelivery]) -> int:
267     if not deliveries:
268         return 0
269     with self._get_connection() as conn:
270         cursor = conn.cursor()
271         video_id = deliveries[0].video_id
272         cursor.execute("DELETE FROM cricket_deliveries WHERE video_id = ?",
273 (video_id,))
274
275         for deliv in deliveries:
276             cursor.execute("""
277             INSERT INTO cricket_deliveries (
278                 video_id, over_num, ball_num, start_time, end_time,
279                 batsman, bowler, runs, wicket, extras
280             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
281             """, (
282                 deliv.video_id,
283                 deliv.over,
284                 deliv.ball,
285                 deliv.start_time,
286                 deliv.end_time,
287                 deliv.batsman,
288                 deliv.bowler,
289                 deliv.runs,
290                 1 if deliv.wicket else 0,
291                 deliv.extras

```

```

292         conn.commit()
293         return len(deliveries)
294
295     def insert_pickleball_rallies(self, rallies: List[PickleballRally]) -> int:
296         if not rallies:
297             return 0
298         with self._get_connection() as conn:
299             cursor = conn.cursor()
300             video_id = rallies[0].video_id
301             cursor.execute("DELETE FROM pickleball_rallies WHERE video_id = ?",
302 (video_id,))
303
304             for rally in rallies:
305                 cursor.execute("""
306                     INSERT INTO pickleball_rallies (
307                         video_id, rally_number, start_time, end_time,
308                         score_before, score_after, shots_count, ending_reason,
309                         last_shot_outcome
310                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
311                     """, (
312                         rally.video_id,
313                         rally.rally_number,
314                         rally.start_time,
315                         rally.end_time,
316                         rally.score_before,
317                         rally.score_after,
318                         rally.shots_count,
319                         rally.ending_reason,
320                         rally.last_shot_outcome
321                     ))
322             conn.commit()
323             return len(rallies)
324
325     def insert_tabletennis_rallies(self, rallies: List[TableTennisRally]) -> int:
326         if not rallies:
327             return 0
328         with self._get_connection() as conn:
329             cursor = conn.cursor()
330             video_id = rallies[0].video_id
331             cursor.execute("DELETE FROM tabletennis_rallies WHERE video_id = ?",
332 (video_id,))
333
334             for rally in rallies:
335                 cursor.execute("""
336                     INSERT INTO tabletennis_rallies (
337                         video_id, rally_number, start_time, end_time,
338                         score_before, score_after, shots_count, ending_reason,
339                         last_shot_outcome
340                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
341                     """, (
342                         rally.video_id,
343                         rally.rally_number,
344                         rally.start_time,
345                         rally.end_time,
346                         rally.score_before,
347                         rally.score_after,
348                         rally.shots_count,
349                         rally.ending_reason,
350                         rally.last_shot_outcome
351                     ))
352             conn.commit()
353             return len(rallies)
354
355     def get_video(self, video_id: str) -> Optional[VideoMetadata]:
356         with self._get_connection() as conn:

```

```

353         cursor = conn.cursor()
354         cursor.execute("SELECT * FROM videos WHERE video_id = ?", (video_id,))
355         row = cursor.fetchone()
356         if row:
357             return VideoMetadata(
358                 video_id=row['video_id'],
359                 sport=SportType(row['sport']),
360                 video_url=row['video_url'],
361                 local_path=row['local_path'],
362                 license_type=LicenseType(row['license_type']),
363                 license_url=row['license_url'],
364                 is_commercial=bool(row['is_commercial']),
365                 status=CurationStatus(row['status']),
366                 match_name=row['match_name'],
367                 fps=row['fps'],
368                 resolution=row['resolution']
369             )
370         return None
371
372     def get_videos_by_status(self, status: CurationStatus, sport: Optional[SportType] =
None) -> List[VideoMetadata]:
373         query = "SELECT * FROM videos WHERE status = ?"
374         params = [status.value]
375         if sport:
376             query += " AND sport = ?"
377             params.append(sport.value)
378
379         videos = []
380         with self._get_connection() as conn:
381             cursor = conn.cursor()
382             cursor.execute(query, params)
383             for row in cursor.fetchall():
384                 videos.append(VideoMetadata(
385                     video_id=row['video_id'],
386                     sport=SportType(row['sport']),
387                     video_url=row['video_url'],
388                     local_path=row['local_path'],
389                     license_type=LicenseType(row['license_type']),
390                     license_url=row['license_url'],
391                     is_commercial=bool(row['is_commercial']),
392                     status=CurationStatus(row['status']),
393                     match_name=row['match_name'],
394                     fps=row['fps'],
395                     resolution=row['resolution']
396                 ))
397         return videos
398
399     def get_status_counts(self) -> List[tuple]:
400         """Return (sport, status, count) rows for the status summary.
401
402         Public read API so callers (e.g. the CLI `status` command) don't have to
403         reach into the private `_get_connection` context manager.
404         """
405         with self._get_connection() as conn:
406             cursor = conn.cursor()
407             cursor.execute(
408                 "SELECT sport, status, COUNT(*) as cnt FROM videos GROUP BY status,
sport"
409             )
410             return [(r["sport"], r["status"], r["cnt"]) for r in cursor.fetchall()]
411
412     def get_badminton_rallies(self, video_id: str) -> List[BadmintonRally]:
413         rallies = []
414         with self._get_connection() as conn:
415             cursor = conn.cursor()

```



```

416         cursor.execute("""
417         SELECT * FROM badminton_rallies
418         WHERE video_id = ?
419         ORDER BY rally_number
420         """, (video_id,))
421         for row in cursor.fetchall():
422             rallies.append(BadmintonRally(
423                 video_id=row['video_id'],
424                 rally_number=row['rally_number'],
425                 start_time=row['start_time'],
426                 end_time=row['end_time'],
427                 score_before=row['score_before'],
428                 score_after=row['score_after'],
429                 shots_count=row['shots_count'],
430                 ending_reason=row['ending_reason'],
431                 last_shot_outcome=row['last_shot_outcome']
432             ))
433         return rallies
434
435     def get_tennis_points(self, video_id: str) -> List[TennisPoint]:
436         points = []
437         with self._get_connection() as conn:
438             cursor = conn.cursor()
439             cursor.execute("""
440             SELECT * FROM tennis_points
441             WHERE video_id = ?
442             ORDER BY point_number
443             """, (video_id,))
444             for row in cursor.fetchall():
445                 points.append(TennisPoint(
446                     video_id=row['video_id'],
447                     point_number=row['point_number'],
448                     start_time=row['start_time'],
449                     end_time=row['end_time'],
450                     set_score=row['set_score'],
451                     game_score=row['game_score'],
452                     server=row['server'],
453                     ending_type=row['ending_type']
454                 ))
455             return points
456
457     def get_cricket_deliveries(self, video_id: str) -> List[CricketDelivery]:
458         deliveries = []
459         with self._get_connection() as conn:
460             cursor = conn.cursor()
461             cursor.execute("""
462             SELECT * FROM cricket_deliveries
463             WHERE video_id = ?
464             ORDER BY over_num, ball_num
465             """, (video_id,))
466             for row in cursor.fetchall():
467                 deliveries.append(CricketDelivery(
468                     video_id=row['video_id'],
469                     over=row['over_num'],
470                     ball=row['ball_num'],
471                     start_time=row['start_time'],
472                     end_time=row['end_time'],
473                     batsman=row['batsman'],
474                     bowler=row['bowler'],
475                     runs=row['runs'],
476                     wicket=bool(row['wicket']),
477                     extras=row['extras']
478                 ))
479             return deliveries
480

```

```

481 def get_pickleball_rallies(self, video_id: str) -> List[PickleballRally]:
482     rallies = []
483     with self._get_connection() as conn:
484         cursor = conn.cursor()
485         cursor.execute("""
486             SELECT * FROM pickleball_rallies
487             WHERE video_id = ?
488             ORDER BY rally_number
489             """, (video_id,))
490         for row in cursor.fetchall():
491             rallies.append(PickleballRally(
492                 video_id=row['video_id'],
493                 rally_number=row['rally_number'],
494                 start_time=row['start_time'],
495                 end_time=row['end_time'],
496                 score_before=row['score_before'],
497                 score_after=row['score_after'],
498                 shots_count=row['shots_count'],
499                 ending_reason=row['ending_reason'],
500                 last_shot_outcome=row['last_shot_outcome']
501             ))
502     return rallies
503
504 def get_tabletennis_rallies(self, video_id: str) -> List[TableTennisRally]:
505     rallies = []
506     with self._get_connection() as conn:
507         cursor = conn.cursor()
508         cursor.execute("""
509             SELECT * FROM tabletennis_rallies
510             WHERE video_id = ?
511             ORDER BY rally_number
512             """, (video_id,))
513         for row in cursor.fetchall():
514             rallies.append(TableTennisRally(
515                 video_id=row['video_id'],
516                 rally_number=row['rally_number'],
517                 start_time=row['start_time'],
518                 end_time=row['end_time'],
519                 score_before=row['score_before'],
520                 score_after=row['score_after'],
521                 shots_count=row['shots_count'],
522                 ending_reason=row['ending_reason'],
523                 last_shot_outcome=row['last_shot_outcome']
524             ))
525     return rallies
526
527 def get_segment_intervals(self, video_id: str, sport: SportType) -> List[tuple]:
528     """Return chronologically-ordered (start_time, end_time) tuples for a video,
529     regardless of sport. This is the sport-agnostic view used to export
530     indexer-ready evaluation/training labels.
531
532     Sports that have no annotation table yet (e.g. tabletennis, pickleball)
533     yield an empty list rather than raising.
534     """
535     mapping = SEGMENT_TABLES.get(sport)
536     if mapping is None:
537         return []
538     table, order_by = mapping # both come from the trusted map above, never user
input
539     with self._get_connection() as conn:
540         cursor = conn.cursor()
541         cursor.execute(
542             f"SELECT start_time, end_time FROM {table} "
543             f"WHERE video_id = ? ORDER BY {order_by}",
544             (video_id,))

```

```

545         )
546         return [(row["start_time"], row["end_time"]) for row in cursor.fetchall()]
547
548     def update_status(self, video_id: str, status: CurationStatus) -> bool:
549         with self._get_connection() as conn:
550             cursor = conn.cursor()
551             cursor.execute("UPDATE videos SET status = ? WHERE video_id = ?",
552 (status.value, video_id))
553             conn.commit()
554             return cursor.rowcount > 0
555
556     def delete_video(self, video_id: str) -> bool:
557         """Deletes a video and cascading child entries."""
558         with self._get_connection() as conn:
559             cursor = conn.cursor()
560             # Execute cascade deletes manually just in case SQLite foreign key support
561 is not active
562             cursor.execute("DELETE FROM badminton_rallies WHERE video_id = ?",
563 (video_id,))
564             cursor.execute("DELETE FROM tennis_points WHERE video_id = ?", (video_id,))
565             cursor.execute("DELETE FROM cricket_deliveries WHERE video_id = ?",
566 (video_id,))
567             cursor.execute("DELETE FROM pickleball_rallies WHERE video_id = ?",
568 (video_id,))
569             cursor.execute("DELETE FROM tabletennis_rallies WHERE video_id = ?",
570 (video_id,))
571             cursor.execute("DELETE FROM videos WHERE video_id = ?", (video_id,))
572             conn.commit()
573             return cursor.rowcount > 0
574
575

```

10. assistant (2026-06-07T06:41:49.901Z)

Now let me check the validator and the CVAT parser to understand validation and report generation:

11. user (2026-06-07T06:41:50.289Z)

```

1     from typing import List, Tuple, Union
2     from src.core.config import load_config
3     from src.core.schemas.base import VideoMetadata, LicenseType
4     from src.core.schemas.badminton import BadmintonRally
5     from src.core.schemas.tennis import TennisPoint
6     from src.core.schemas.cricket import CricketDelivery
7     from src.core.schemas.pickleball import PickleballRally
8     from src.core.schemas.tabletennis import TableTennisRally
9
10    class DatasetValidator:
11        def __init__(self, config_path: str = "config/config.yaml"):
12            self.config_path = config_path
13            self.commercial_licenses = self._load_licenses_whitelist()
14
15        def _load_licenses_whitelist(self) -> List[str]:
16            # load_config returns {} on a missing/unreadable file, so the default list
17            # below is the single fallback (keep in sync with config.yaml).
18            config = load_config(self.config_path)
19            return config.get(
20                'commercial_licenses',
21                ["MIT", "Apache-2.0", "BSD", "CC0", "CC-BY", "Public-Domain",
22 "Proprietary"],
23            )
24
25        def check_license_commercial(self, license_type: Union[LicenseType, str]) -> bool:
26            """Determines if a license type is safe for commercial use based on the
27 configuration whitelist."""

```

```

26         val = license_type.value if isinstance(license_type, LicenseType) else
license_type
27         return val in self.commercial_licenses
28
29     def validate_badminton_rallies(self, rallies: List[BadmintonRally]) -> Tuple[bool,
List[str]]:
30         """
31         Validates badminton rallies:
32         - Sorted chronologically.
33         - Start time < End time (handled by Pydantic, but double checked).
34         - No temporal overlap between sequential rallies.
35         """
36         errors = []
37         if not rallies:
38             return True, errors
39
40         # Sort by rally number to check progression
41         sorted_rallies = sorted(rallies, key=lambda r: r.rally_number)
42
43         for i in range(len(sorted_rallies)):
44             r = sorted_rallies[i]
45             if r.start_time >= r.end_time:
46                 errors.append(f"Rally {r.rally_number}: start_time ({r.start_time}) >=
end_time ({r.end_time})")
47
48             if i > 0:
49                 prev = sorted_rallies[i - 1]
50                 if r.start_time < prev.end_time:
51                     errors.append(
52                         f"Rally overlap detected between Rally {prev.rally_number} "
53                         f"(ends at {prev.end_time}s) and Rally {r.rally_number} "
54                         f"(starts at {r.start_time}s)"
55                     )
56
57         return len(errors) == 0, errors
58
59     def validate_tennis_points(self, points: List[TennisPoint]) -> Tuple[bool,
List[str]]:
60         errors = []
61         if not points:
62             return True, errors
63
64         sorted_pts = sorted(points, key=lambda p: p.point_number)
65         for i in range(len(sorted_pts)):
66             pt = sorted_pts[i]
67             if pt.start_time >= pt.end_time:
68                 errors.append(f"Point {pt.point_number}: start ({pt.start_time}) >= end
({pt.end_time})")
69
70             if i > 0:
71                 prev = sorted_pts[i - 1]
72                 if pt.start_time < prev.end_time:
73                     errors.append(
74                         f"Point overlap detected: Point {prev.point_number} (ends
{prev.end_time}s) "
75                         f"and Point {pt.point_number} (starts {pt.start_time}s)"
76                     )
77         return len(errors) == 0, errors
78
79     def validate_cricket_deliveries(self, deliveries: List[CricketDelivery]) ->
Tuple[bool, List[str]]:
80         errors = []
81         if not deliveries:
82             return True, errors
83

```



```
143         )
144         return len(errors) == 0, errors
145
```

12. assistant (2026-06-07T06:41:52.342Z)

Now let me check the CVAT parser which handles issue detection and report generation:

13. user (2026-06-07T06:41:52.905Z)

```
1     """Adapter: CVAT rally exports -> our canonical per-rally records.
2
3     Why this exists
4     -----
5     Our annotation brief (`docs/annotation/ROORA_BRIEF.md`) asks vendors for a
6     per-rally CSV that imports directly via curate import-local. A vendor may
7     instead do the work in CVAT and hand back the CVAT export: per-frame bounding
8     boxes, with the rally fields (rally_number, shots_count,
9     ending_reason ...) carried as CVAT object attributes on each box. The box
10    geometry is irrelevant to us ? it is just a carrier the rater attached the rally
11    metadata to.
12
13    This module is the ingestion-side adapter that absorbs that vendor format so the
14    modeling repo never has to. It does two things, both deliberately on our side
15    so we fix mechanical problems rather than renegotiating them:
16
17    1. Recompute time from the frame range. A rally's interval is taken from the
18    first/last frame a box for that rally appears on:
19    seconds = frame / fps at the video's true fps. We ignore any
20    start_time/end_time attributes the vendor wrote ? a CVAT job that
21    samples every step frames and assumes 30 fps yields systematically wrong
22    times (we have seen start_time == frame / (step * 30), i.e. off by the
23    sampling step at the wrong fps). The frame index is reliable; the vendor's
24    seconds are not.
25
26    2. Surface ? never silently fix ? content defects. Overlapping rallies,
27    zero-duration rallies, ending_reason values outside the agreed set, and
28    too-coarse sampling are reported as :class:`Issue`s. Guessing a corrected
29    rally boundary would poison ground truth, so that stays a human/vendor call.
30
31    Supported input: the single-file CVAT XML export, in either image mode
32    (<image><box>...) or track/interpolation mode (<track><box frame=...>).
33    The two PASCAL-VOC / COCO sibling exports encode the same data and can be added
34    later if a vendor sends only those.
35    """
36    from __future__ import annotations
37
38    import os
39    import xml.etree.ElementTree as ET
40    from collections import Counter
41    from dataclasses import dataclass, field
42    from typing import Any, Dict, List, Optional, Tuple
43
44    from src.core.parsing import safe_int
45    from src.core.downloader import probe_resolution_fps
46
47    # The agreed cross-sport ending_reason vocabulary for net-separated sports.
48    # Kept here as a soft check only (a warning, never a hard failure) so this
49    # adapter does not depend on the annotation-spec wording.
50    DEFAULT_ALLOWED_REASONS = {
51        "winner",
52        "forced_error",
53        "unforced_error",
54        "service_fault",
55        "let",
```

```

56         "other",
57     }
58
59     # The observational outcome of the last shot ? orthogonal to ending_reason
60     # (winner can be 'in'; an error can be 'net' or 'out'; service_fault/let -> 'n_a').
61     DEFAULT_ALLOWED_OUTCOMES = {"in", "net", "out", "n_a"}
62
63     # CVAT attribute names we map onto canonical columns. The vendor's CVAT label
64     # schema has used suffixed names like "score_before (A-B)"; we match leniently.
65     _ATTR_ALIASES = {
66         "rally_number": ("rally_number",),
67         "shots_count": ("shots_count",),
68         "ending_reason": ("ending_reason",),
69         "last_shot_outcome": ("last_shot_outcome", "outcome", "last_shot_outcome
(in/net/out)"),
70         "score_before": ("score_before", "score_before (a-b)"),
71         "score_after": ("score_after", "score_after (a-b)"),
72         "notes": ("notes", "note"),
73     }
74
75     CANONICAL_COLUMNS = [
76         "rally_number", "start_mmss", "end_mmss", "start_time", "end_time",
77         "shots_count", "ending_reason", "last_shot_outcome", "score_before", "score_after",
"notes",
78     ]
79
80
81     @dataclass
82     class RallyRecord:
83         """One rally recovered from the CVAT export, time re-derived from frames."""
84         rally_number: int
85         start_time: float          # seconds, = frame_min / fps
86         end_time: float            # seconds, = frame_max / fps (see padding note)
87         shots_count: Optional[int]
88         ending_reason: Optional[str]
89         last_shot_outcome: Optional[str]
90         score_before: Optional[str]
91         score_after: Optional[str]
92         notes: Optional[str]
93         frame_min: int
94         frame_max: int
95         n_boxes: int
96         vendor_start_time: Optional[str] = None    # the (discarded) attribute value
97         vendor_end_time: Optional[str] = None
98
99         @property
100         def duration(self) -> float:
101             return self.end_time - self.start_time
102
103
104     @dataclass
105     class Issue:
106         """A defect found in the delivery. ``blocker`` issues stop a clean import."""
107         severity: str    # 'blocker' | 'high' | 'medium' | 'low'
108         code: str
109         message: str
110         rally_number: Optional[int] = None
111
112
113     @dataclass
114     class ConversionResult:
115         records: List[RallyRecord]
116         issues: List[Issue]
117         fps: float
118         step: int

```

```

119     meta: Dict[str, Any] = field(default_factory=dict)
120
121     @property
122     def n_blockers(self) -> int:
123         return sum(1 for i in self.issues if i.severity == "blocker")
124
125
126     def _mmss(seconds: float) -> str:
127         """Format seconds as ``m:ss.mmm`` (matches the brief's mm:ss columns)."""
128         m = int(seconds // 60)
129         s = seconds - m * 60
130         return f"{m}:{s:06.3f}"
131
132
133     def _attr_lookup(attrs: Dict[str, str], canonical: str) -> Optional[str]:
134         for alias in _ATTR_ALIASES[canonical]:
135             if alias in attrs and attrs[alias] != "":
136                 return attrs[alias]
137         return None
138
139
140     def parse_cvat_boxes(xml_path: str) -> Tuple[Dict[str, Any], List[Dict[str, Any]]]:
141         """Parse a CVAT XML export into (meta, boxes).
142
143         ``boxes`` is a flat list of ``{"frame": int, "attrs": {name: value}}`` ? one
144         entry per visible box, from either image-mode or track-mode exports.
145         ``meta`` carries ``step`` (frame_filter) and ``stop_frame`` when present.
146         """
147         if not os.path.exists(xml_path):
148             raise FileNotFoundError(f"CVAT export not found: {xml_path}")
149
150         root = ET.parse(xml_path).getroot()
151         meta: Dict[str, Any] = {}
152
153         job = root.find("./job")
154         if job is not None:
155             stop = job.findtext("stop_frame")
156             if stop is not None and stop.strip().isdigit():
157                 meta["stop_frame"] = int(stop.strip())
158             ff = job.findtext("frame_filter") or ""
159             if "step=" in ff:
160                 try:
161                     meta["step"] = int(ff.split("step=")[1].split()[0])
162                 except (ValueError, IndexError):
163                     pass
164
165         def _attrs(el: ET.Element) -> Dict[str, str]:
166             out: Dict[str, str] = {}
167             for a in el.findall("attribute"):
168                 out[(a.get("name") or "").strip().lower()] = (a.text or "").strip()
169             return out
170
171         boxes: List[Dict[str, Any]] = []
172
173         # Image mode: <image id=frame> ... <box>...</box>
174         for img in root.findall("image"):
175             try:
176                 frame = int(img.get("id"))
177             except (TypeError, ValueError):
178                 continue
179             for box in img.findall("box"):
180                 boxes.append({"frame": frame, "attrs": _attrs(box)})
181
182         # Track mode: <track> ... <box frame=N outside=0>...</box>
183         for track in root.findall("track"):

```



```

184         for box in track.findall("box"):
185             if box.get("outside") == "1":
186                 continue # a track's terminating keyframe carries no real label
187             try:
188                 frame = int(box.get("frame"))
189             except (TypeError, ValueError):
190                 continue
191             boxes.append({"frame": frame, "attrs": _attrs(box)})
192
193     return meta, boxes
194
195
196     def _infer_step(frames: List[int]) -> int:
197         """Most common positive gap between consecutive distinct sampled frames."""
198         uniq = sorted(set(frames))
199         gaps = [b - a for a, b in zip(uniq, uniq[1:]) if b - a > 0]
200         if not gaps:

```

14. assistant (2026-06-07T06:41:53.978Z)

Let me continue reading the CVAT parser:

15. user (2026-06-07T06:41:54.523Z)

```

200         if not gaps:
201             return 1
202         return Counter(gaps).most_common(1)[0][0]
203
204
205     def build_records(boxes: List[Dict[str, Any]], fps: float) -> List[RallyRecord]:
206         """Group boxes by rally_number and derive each rally's frame range -> seconds."""
207         if fps is None or fps <= 0:
208             raise ValueError("fps must be a positive number to convert frames to seconds")
209
210         by_rally: Dict[int, List[Dict[str, Any]]] = {}
211         for b in boxes:
212             rn = safe_int(_attr_lookup(b["attrs"], "rally_number"), None)
213             if rn is None:
214                 continue
215             by_rally.setdefault(rn, []).append(b)
216
217         records: List[RallyRecord] = []
218         for rn in sorted(by_rally):
219             items = by_rally[rn]
220             frames = sorted(b["frame"] for b in items)
221             attrs = items[0]["attrs"]
222             fmin, fmax = frames[0], frames[-1]
223             records.append(RallyRecord(
224                 rally_number=rn,
225                 start_time=fmin / fps,
226                 end_time=fmax / fps,
227                 shots_count=safe_int(_attr_lookup(attrs, "shots_count"), None),
228                 ending_reason=_attr_lookup(attrs, "ending_reason"),
229                 last_shot_outcome=_attr_lookup(attrs, "last_shot_outcome"),
230                 score_before=_attr_lookup(attrs, "score_before"),
231                 score_after=_attr_lookup(attrs, "score_after"),
232                 notes=_attr_lookup(attrs, "notes"),
233                 frame_min=fmin,
234                 frame_max=fmax,
235                 n_boxes=len(frames),
236                 vendor_start_time=attrs.get("start_time"),
237                 vendor_end_time=attrs.get("end_time"),
238             ))
239         return records
240

```

```

241
242 def detect_issues(records: List[RallyRecord],
243                 stop_frame: Optional[int] = None,
244                 allowed_reasons: Optional[set] = None,
245                 allowed_outcomes: Optional[set] = None,
246                 max_gap_s: float = 15.0) -> List[Issue]:
247     """Find the defects that block or degrade a clean ground-truth import.
248
249     Note: we deliberately do NOT flag the sampling density (step/fps). A boundary
250     slack of ~0.5s is acceptable for this pipeline because the downstream
251     highlight compiler pads every clip on output (badminton-highlight-indexer,
252     backend/stitcher/compiler.py: begin_padding=0.5s, end_padding=1.0s), which
253     absorbs the sampling uncertainty when highlights are trimmed.
254     """
255     allowed = allowed_reasons if allowed_reasons is not None else
256     DEFAULT_ALLOWED_REASONS
257     allowed_out = allowed_outcomes if allowed_outcomes is not None else
258     DEFAULT_ALLOWED_OUTCOMES
259     issues: List[Issue] = []
260
261     ordered = sorted(records, key=lambda r: r.rally_number)
262     for i, r in enumerate(ordered):
263         if r.end_time <= r.start_time:
264             issues.append(Issue(
265                 "blocker", "zero_duration",
266                 f"Rally {r.rally_number}: end <= start "
267                 f"({frames {r.frame_min}-{r.frame_max}}); single-sample or stray box.",
268                 r.rally_number))
269
270         if r.ending_reason is not None and r.ending_reason not in allowed:
271             issues.append(Issue(
272                 "medium", "ending_reason_vocab",
273                 f"Rally {r.rally_number}: ending_reason '{r.ending_reason}' "
274                 f"not in the agreed set {sorted(allowed)}.",
275                 r.rally_number))
276
277         if r.last_shot_outcome is not None and r.last_shot_outcome not in allowed_out:
278             issues.append(Issue(
279                 "low", "outcome_vocab",
280                 f"Rally {r.rally_number}: last_shot_outcome '{r.last_shot_outcome}' "
281                 f"not in {sorted(allowed_out)}.",
282                 r.rally_number))
283
284         # Inflated span: a long rally with implausibly few shots usually means a
285         # stray/duplicated box stretched the frame range past the real ending.
286         if r.shots_count and r.duration >= 5.0:
287             sps = r.shots_count / r.duration
288             if sps < 0.3:
289                 issues.append(Issue(
290                     "medium", "shot_density",
291                     f"Rally {r.rally_number}: {r.shots_count} shots over "
292                     f"{r.duration:.1f}s ({sps:.2f}/s) is implausibly low ? the span "
293                     f"may be inflated by a stray box.",
294                     r.rally_number))
295
296         if stop_frame is not None and r.frame_max > stop_frame:
297             issues.append(Issue(
298                 "high", "frame_out_of_range",
299                 f"Rally {r.rally_number}: frame {r.frame_max} exceeds the video's "
300                 f"last frame {stop_frame}.",
301                 r.rally_number))
302
303         if i > 0:
304             prev = ordered[i - 1]
305             if r.start_time < prev.end_time:

```

```

304         issues.append(Issue(
305             "blocker", "overlap",
306             f"Rally {prev.rally_number} (ends {prev.end_time:.2f}s) overlaps "
307             f"Rally {r.rally_number} (starts {r.start_time:.2f}s) by "
308             f"{prev.end_time - r.start_time:.2f}s ? rallies must be disjoint.",
309             r.rally_number))
310     elif r.start_time - prev.end_time > max_gap_s:
311         # A long unlabeled stretch between two rallies is a cheap, label-only
312         # second opinion for the worst error (a missed rally). It won't catch a
313         # rally MERGED into its neighbour's span (shot_density/inflated catches
314         # that), but it does catch an omitted rally that left a visible hole.
315         gap = r.start_time - prev.end_time
316         issues.append(Issue(
317             "medium", "possible_missed_rally",
318             f"{gap:.1f}s of unlabeled time between Rally {prev.rally_number} "
319             f"(ends {prev.end_time:.1f}s) and Rally {r.rally_number} "
320             f"(starts {r.start_time:.1f}s) ? check for an unlabeled rally in the
gap.",
321             r.rally_number))
322     return issues
323
324
325 def convert(xml_path: str,
326            fps: Optional[float] = None,
327            video_path: Optional[str] = None,
328            step: Optional[int] = None,
329            allowed_reasons: Optional[set] = None) -> ConversionResult:
330     """Full pipeline: parse CVAT XML -> records (+ issues) at the true fps.
331
332     ``fps`` resolution order: explicit ``fps`` arg > ffprobe of ``video_path`` >
333     the CVAT meta (none) -> error. We never fall back to a guessed 30 fps, since
334     that silent guess is exactly the bug this adapter exists to undo.
335     """
336     meta, boxes = parse_cvat_boxes(xml_path)
337
338     if fps is None and video_path:
339         _, _, probed = probe_resolution_fps(video_path)
340         fps = probed
341     if fps is None or fps <= 0:
342         raise ValueError(
343             "Could not determine fps. Pass fps=... or a readable video_path so "
344             "frames can be converted to seconds (seconds = frame / fps).")
345
346     if step is None:
347         step = meta.get("step") or _infer_step([b["frame"] for b in boxes])
348
349     records = build_records(boxes, fps)
350     issues = detect_issues(records,
351                            stop_frame=meta.get("stop_frame"),
352                            allowed_reasons=allowed_reasons)
353     return ConversionResult(records=records, issues=issues, fps=fps, step=step,
meta=meta)
354
355
356 def write_canonical_csv(records: List[RallyRecord], out_path: str) -> None:
357     """Write records as the canonical per-rally CSV that ``import-local`` reads."""
358     import csv
359     os.makedirs(os.path.dirname(os.path.abspath(out_path)), exist_ok=True)
360     with open(out_path, "w", newline="", encoding="utf-8") as f:
361         w = csv.writer(f)
362         w.writerow(CANONICAL_COLUMNS)
363         for r in sorted(records, key=lambda x: x.rally_number):
364             w.writerow([
365                 r.rally_number,
366                 _mmss(r.start_time), _mmss(r.end_time),

```

```

367         f"{r.start_time:.3f}", f"{r.end_time:.3f}",
368         r.shots_count if r.shots_count is not None else "",
369         r.ending_reason or "",
370         r.last_shot_outcome or "",
371         r.score_before or "", r.score_after or "", r.notes or "",
372     ])
373
374
375 def render_issue_report(result: ConversionResult, source: str) -> str:
376     """A human-readable issue report (for review / sending back to the vendor)."""
377     lines = [
378         f"CVAT rally conversion report ? {source}",
379         f"  fps used          : {result.fps:.3f}",
380         f"  sampling step      : {result.step} frames (~{result.step / result.fps:.2f}s)",
381         f"  rallies parsed     : {len(result.records)}",
382         f"  issues              : {len(result.issues)} ({result.n_blockers} blocker)",
383         "",
384     ]
385     if not result.issues:
386         lines.append("No issues detected.")
387     else:
388         for sev in ("blocker", "high", "medium", "low"):
389             group = [i for i in result.issues if i.severity == sev]
390             if not group:
391                 continue
392             lines.append(f"[{sev.upper()}]")
393             for i in group:
394                 lines.append(f"  - ({i.code}) {i.message}")
395             lines.append("")
396     return "\n".join(lines)
397
398
399 # ----- #
400 # Validation reporting ? used by `curate validate-delivery`. #
401 # These turn a (records, issues) pair into the artifacts a human reviews: #
402 # a machine summary (JSON), a human report (md), and an annotatable sheet. #
403 # ----- #
404
405 def _parse_secs(value: Optional[str]) -> Optional[float]:
406     """Decimal seconds or mm:ss / hh:mm:ss -> float; None if blank/unparseable."""
407     s = (value or "").strip()
408     if not s:
409         return None
410     if ":" in s:
411         try:
412             total = 0.0
413             for part in s.split(":"):
414                 total = total * 60 + float(part)
415             return total
416         except ValueError:
417             return None
418     try:
419         return float(s)
420     except ValueError:
421         return None
422
423
424 def load_canonical_csv(csv_path: str) -> List[RallyRecord]:
425     """Load a canonical per-rally CSV into RallyRecords, so a *CSV* delivery (not
426     just a CVAT export) can be validated. Decimal start/end preferred, mm:ss
427     fallback; rows with no usable start/end are skipped. (frame_* are -1 here, as a
428     CSV carries no frame info ? only the frame-derived checks become inapplicable.)
429     """
430     import csv as _csv
431     if not os.path.exists(csv_path):

```

```

432         raise FileNotFoundError(f"CSV not found: {csv_path}")
433     records: List[RallyRecord] = []
434     with open(csv_path, newline="", encoding="utf-8") as f:
435         reader = _csv.DictReader(f)
436         reader.fieldnames = [(n or "").strip().lower() for n in (reader.fieldnames or
437 [])]
438         for raw in reader:
439             row = {(k or "").strip().lower(): v for k, v in raw.items()}
440             rn = safe_int(row.get("rally_number"), None)
441             if rn is None:
442                 continue
443             start = _parse_secs(row.get("start_time")) or
444 _parse_secs(row.get("start_mmss"))
445             end = _parse_secs(row.get("end_time")) or _parse_secs(row.get("end_mmss"))
446             if start is None or end is None:
447                 continue
448             records.append(RallyRecord(
449                 rally_number=rn, start_time=start, end_time=end,
450                 shots_count=safe_int(row.get("shots_count"), None),
451                 ending_reason=(row.get("ending_reason") or "").strip() or None,
452                 last_shot_outcome=(row.get("last_shot_outcome") or "").strip() or None,
453                 score_before=(row.get("score_before") or "").strip() or None,
454                 score_after=(row.get("score_after") or "").strip() or None,
455                 notes=(row.get("notes") or "").strip() or None,
456                 frame_min=-1, frame_max=-1, n_boxes=0,
457             ))
458     return records
459
460 def build_summary(records: List[RallyRecord], issues: List[Issue],
461                 source: str = "", fps: Optional[float] = None) -> Dict[str, Any]:
462     """Machine-readable validation summary (JSON / CI)."""
463     by_sev = {s: [i for i in issues if i.severity == s]
464               for s in ("blocker", "high", "medium", "low")}
465     flagged = sorted({i.rally_number for i in issues if i.rally_number is not None})
466     return {
467         "source": source,
468         "fps": fps,
469         "n_rallies": len(records),
470         "n_issues": len(issues),
471         "counts_by_severity": {s: len(v) for s, v in by_sev.items()},
472         "n_blockers": len(by_sev["blocker"]),
473         "passed": len(by_sev["blocker"]) == 0,
474         "flagged_rallies": flagged,
475         "issues": [{"severity": i.severity, "code": i.code,
476                    "rally_number": i.rally_number, "message": i.message} for i in
477 issues],
478     }
479
480 def render_validation_md(records: List[RallyRecord], issues: List[Issue], source: str)
481 -> str:
482     """Human validation report: scoreboard + issues by severity + rallies to review."""
483     n = len(records)
484     by_sev = {s: [i for i in issues if i.severity == s]
485               for s in ("blocker", "high", "medium", "low")}
486     flagged = sorted({i.rally_number for i in issues if i.rally_number is not None})
487     clean = n - len(flagged)
488     lines = [
489         f"# Delivery validation ? {source}",
490         "",
491         f"- rallies: **{n}**",
492         f"- issues: **{len(issues)}** (blocker {len(by_sev['blocker'])}, "
493         f"high {len(by_sev['high'])}, medium {len(by_sev['medium'])}, low
494 {len(by_sev['low'])})",

```

```

492         f"- structurally-clean rallies: **{clean}/{n}**"
493         + (f" ({100 * clean // n}%)" if n else ""),
494         f"- gate: **{'PASS' if not by_sev['blocker'] else 'BLOCKED'}**",
495         "",
496     ]
497     if issues:
498         for sev in ("blocker", "high", "medium", "low"):
499             grp = by_sev[sev]
500             if not grp:
501                 continue
502             lines.append(f"## {sev.upper()}")
503             lines += [f"- ({i.code}) {i.message}" for i in grp]
504             lines.append("")
505     else:
506         lines.append("No issues detected. ?\n")
507     if flagged:
508         lines.append(f"## Rallies to review: {flagged}")
509     return "\n".join(lines)
510
511
512 def write_review_csv(records: List[RallyRecord], issues: List[Issue], out_path: str) ->
None:
513     """Auto-generate the annotatable manual-verification sheet (blank `true_` cols)."""
514     import csv
515     flags: Dict[int, List[str]] = {}
516     for i in issues:
517         if i.rally_number is not None:
518             flags.setdefault(i.rally_number, []).append(i.code)
519     os.makedirs(os.path.dirname(os.path.abspath(out_path)), exist_ok=True)
520     cols = ["rally_number", "start_mmss", "end_mmss", "dur_s", "shots_count",
521            "ending_reason", "last_shot_outcome", "auto_flags",
522            "VERIFIED(Y/N)", "true_start_mmss", "true_end_mmss",
523            "true_ending_reason", "true_last_shot_outcome", "reviewer_comment"]
524     with open(out_path, "w", newline="", encoding="utf-8") as f:
525         w = csv.writer(f)
526         w.writerow(cols)
527         for r in sorted(records, key=lambda x: x.rally_number):
528             w.writerow([
529                 r.rally_number, _mmss(r.start_time), _mmss(r.end_time),
530                 f"{r.duration:.1f}", r.shots_count if r.shots_count is not None else "",
531                 r.ending_reason or "", r.last_shot_outcome or "",
532                 ";".join(flags.get(r.rally_number, [])),
533                 "", "", "", "", "", "", "",
534             ])
535

```

16. assistant (2026-06-07T06:41:56.125Z)

Now let me check the data schemas to understand the canonical data shapes:

17. user (2026-06-07T06:41:56.408Z)

```

1     from pydantic import BaseModel, Field
2     from typing import Optional
3     from enum import Enum
4
5     class SportType(str, Enum):
6         BADMINTON = "badminton"
7         TENNIS = "tennis"
8         TABLE_TENNIS = "tabletennis"
9         PICKLEBALL = "pickleball"
10        CRICKET = "cricket"
11
12    class LicenseType(str, Enum):
13        MIT = "MIT"

```

```

14     APACHE_2 = "Apache-2.0"
15     BSD = "BSD"
16     CC0 = "CC0"
17     CC_BY = "CC-BY"
18     CC_BY_NC = "CC-BY-NC"
19     CC_BY_NC_SA = "CC-BY-NC-SA"
20     PUBLIC_DOMAIN = "Public-Domain"
21     PROPRIETARY = "Proprietary"
22     UNKNOWN = "Unknown"
23
24     class CurationStatus(str, Enum):
25         STAGING = "staging"
26         CURATED = "curated"
27         REJECTED = "rejected"
28
29     class VideoMetadata(BaseModel):
30         video_id: str = Field(..., description="Unique alphanumeric identifier for the
video")
31         sport: SportType
32         video_url: Optional[str] = Field(None, description="Remote URL of the video (if
available)")
33         local_path: Optional[str] = Field(None, description="Local path to video file (if
downloaded or custom)")
34         license_type: LicenseType
35         license_url: Optional[str] = None
36         is_commercial: bool = Field(False, description="Flag indicating if the license
allows commercial use")
37         status: CurationStatus = CurationStatus.STAGING
38         match_name: str = Field(..., description="Name of the match/event")
39         fps: Optional[float] = Field(None, description="Frames per second")
40         resolution: Optional[str] = Field(None, description="Resolution string (e.g. 1080p,
720p)")
41

```

18. assistant (2026-06-07T06:41:57.610Z)

Now let me check the sport-specific schemas:

19. user (2026-06-07T06:41:58.097Z)

```

src/core/schemas/badminton.py
src/core/schemas/base.py
src/core/schemas/cricket.py
src/core/schemas/pickleball.py
src/core/schemas/tabletennis.py
src/core/schemas/tennis.py

```

20. assistant (2026-06-07T06:41:59.150Z)

Let me read one of the sport schemas to understand the pattern:

21. user (2026-06-07T06:41:59.326Z)

```

1     from pydantic import BaseModel, Field, model_validator
2     from typing import Optional
3
4     class BadmintonRally(BaseModel):
5         video_id: str
6         rally_number: int = Field(..., description="Chronological rally index in the match")
7         start_time: float = Field(..., description="Start of rally in seconds")
8         end_time: float = Field(..., description="End of rally in seconds")
9         score_before: Optional[str] = Field(None, description="Score before the rally (e.g.
'0-0')")
10        score_after: Optional[str] = Field(None, description="Score after the rally (e.g.

```

```

'1-0'))
11         shots_count: Optional[int] = Field(None, description="Number of strokes in the
rally")
12         ending_reason: Optional[str] = Field(None, description="Why the rally ended:
'winner' | 'forced_error' | 'unforced_error' | 'service_fault' | 'let' | 'other'")
13         last_shot_outcome: Optional[str] = Field(None, description="Where the last shot
landed: 'in' | 'net' | 'out' | 'n_a' (n_a for service_fault/let)")
14
15         @model_validator(mode='after')
16         def validate_timestamps(self) -> 'BadmintonRally':
17             if self.start_time < 0:
18                 raise ValueError("start_time must be non-negative")
19             if self.end_time <= self.start_time:
20                 raise ValueError("end_time must be strictly greater than start_time")
21             return self
22

```

22. assistant (2026-06-07T06:42:00.402Z)

Let me check the tests directory to see what exists:

23. user (2026-06-07T06:42:00.773Z)

```

tests/test_cli.py
tests/test_config.py
tests/test_cvat_parser.py
tests/test_db.py
tests/test_downloader.py
tests/test_local_import.py
tests/test_parser.py
tests/test_parsing.py
tests/test_schemas.py
tests/test_validator.py

```

24. assistant (2026-06-07T06:42:03.639Z)

Now let me check the ingest.py file to see if there are additional ingestion commands:

25. user (2026-06-07T06:42:03.789Z)

```

1     import argparse
2     import sys
3     from src.core.config import load_config, get_database_path
4     from src.core.db import SportsDatabase
5     from src.parsers.badminton.coachai import CoachAIBadmintonParser
6
7     def main():
8         parser = argparse.ArgumentParser(description="Ingest external datasets into the
curation staging area")
9         parser.add_argument("--sport", required=True, choices=["badminton"], help="Sport
type")
10        parser.add_argument("--parser", required=True, choices=["coachai"], help="Parser
type to use")
11        parser.add_argument("--file-path", required=True, help="Path to raw dataset file")
12        parser.add_argument("--license", default="MIT", help="Dataset license type")
13        parser.add_argument("--video-url", default="https://youtube.com/watch?v=example",
help="URL to the video")
14        parser.add_argument("--match-name", help="Name of the match")
15        parser.add_argument("--video-id", help="Video unique identifier")
16
17        args = parser.parse_args()
18
19        # Load db path from the central config (falls back to the default).
20        # NOTE: this importer is intentionally badminton/CoachAI-only for now; other

```



```

21     # sports are populated via `curate import-local` or vendor golden sets.
22     db = SportsDatabase(get_database_path(load_config()))
23
24     if args.sport == "badminton" and args.parser == "coachai":
25         match_parser = CoachAIBadmintonParser()
26         try:
27             video_meta, rallies = match_parser.parse(
28                 args.file_path,
29                 video_id=args.video_id,
30                 video_url=args.video_url,
31                 license=args.license,
32                 match_name=args.match_name
33             )
34
35             # Insert into database (by default parsed entries are marked as staging)
36             db.insert_video(video_meta)
37             count = db.insert_badminton_rallies(rallies)
38
39             print(f"Successfully ingested match '{video_meta.match_name}' into
staging:")
40             print(f" - Video ID: {video_meta.video_id}")
41             print(f" - Total Rallies: {count}")
42             print(f" - License: {video_meta.license_type.value}")
43             print(f" - Commercial safe: {video_meta.is_commercial}")
44         except Exception as e:
45             print(f"Error parsing file: {e}")
46             sys.exit(1)
47
48     if __name__ == "__main__":
49         main()
50

```

26. assistant (2026-06-07T06:42:04.892Z)

Now let me check the config to understand how everything ties together:

27. user (2026-06-07T06:42:05.120Z)

```

1     """Central configuration loader.
2
3     A single place to read ``config/config.yaml`` so modules don't each re-open the
4     file with slightly different fallbacks. Returns ``{}`` when the file is missing or
5     unreadable (callers apply their own defaults), matching the prior per-module
6     behaviour but without the duplicated boilerplate and broad excepts.
7     """
8     import os
9     from typing import Any, Dict
10
11     import yaml
12
13     DEFAULT_CONFIG_PATH = "config/config.yaml"
14     DEFAULT_DB_PATH = "data/sports_metadata.db"
15
16
17     def load_config(config_path: str = DEFAULT_CONFIG_PATH) -> Dict[str, Any]:
18         """Load a YAML config into a dict; return {} if missing/unreadable/empty."""
19         if not os.path.exists(config_path):
20             return {}
21         try:
22             with open(config_path, "r", encoding="utf-8") as f:
23                 return yaml.safe_load(f) or {}
24         except (OSError, yaml.YAMLError):
25             return {}
26
27

```

```

28     def get_database_path(config: Dict[str, Any]) -> str:
29         """Resolve the SQLite database path from a loaded config, with a default."""
30         return config.get("database_path", DEFAULT_DB_PATH)
31

```

28. assistant (2026-06-07T06:42:06.307Z)

Let me check the parsing utilities to understand how timestamps and frames are handled:

29. user (2026-06-07T06:42:06.623Z)

```

1     """Shared parsing helpers used by the CLI importer and dataset parsers.
2
3     Previously these were duplicated as private ``_safe_int`` / ``_safe_float`` /
4     ``_parse_time_string`` in both ``src/cli/curate.py`` and the CoachAI parser.
5     """
6     from typing import Any, Optional
7
8
9     def safe_int(value: Any, default: Optional[int] = 0) -> Optional[int]:
10         """Parse an int from a possibly-empty/float/None value; return default on
failure."""
11         if value is None:
12             return default
13         s = str(value).strip()
14         if not s:
15             return default
16         try:
17             return int(float(s))
18         except (TypeError, ValueError):
19             return default
20
21
22     def safe_float_required(value: Any) -> float:
23         """Parse a float; raise ValueError if missing/empty (caller treats it as
required)."""
24         if value is None:
25             raise ValueError("missing required numeric value")
26         s = str(value).strip()
27         if not s:
28             raise ValueError("missing required numeric value")
29         return float(s)
30
31
32     def parse_time_string(time_str: Any) -> float:
33         """Convert 'HH:MM:SS' / 'MM:SS' / float-strings to float seconds; 0.0 if
blank/invalid."""
34         if not time_str:
35             return 0.0
36         s = str(time_str).strip()
37         if ":" in s:
38             parts = s.split(":")
39             try:
40                 if len(parts) == 3:
41                     return float(parts[0]) * 3600 + float(parts[1]) * 60 + float(parts[2])
42                 if len(parts) == 2:
43                     return float(parts[0]) * 60 + float(parts[1])
44             except ValueError:
45                 return 0.0
46         try:
47             return float(s)
48         except ValueError:
49             return 0.0
50

```

30. assistant (2026-06-07T06:42:07.869Z)

Now let me look at one of the test files to understand the test patterns:

31. user (2026-06-07T06:42:08.162Z)

```
1      """End-to-end coverage for the SportsDataCLI commands.
2
3      Each test runs against a throwaway config + SQLite DB in tmp_path, so the real
4      data/ directory is never touched. Network (yt-dlp) and the browser are mocked.
5      """
6      import csv as _csv
7      import json
8      from types import SimpleNamespace
9      from unittest import mock
10
11     import pytest
12     import yaml
13
14     from src.cli.curate import SportsDataCLI
15     from src.core.schemas.base import VideoMetadata, SportType, LicenseType, CurationStatus
16     from src.core.schemas.badminton import BadmintonRally
17
18
19     @pytest.fixture
20     def cli(tmp_path):
21         cfg = {
22             "database_path": str(tmp_path / "test.db"),
23             "video_storage": {
24                 "local_dir": str(tmp_path / "videos"),
25                 "max_download_size_mb": 100,
26                 "auto_download": False,
27                 "max_height": None,
28             },
29             "commercial_licenses": ["MIT", "Apache-2.0", "BSD", "CC0", "CC-BY", "Public-
Domain", "Proprietary"],
30             "supported_sports": ["badminton", "tennis", "tabletennis", "pickleball",
"cricket"],
31         }
32         cfg_path = tmp_path / "config.yaml"
33         cfg_path.write_text(yaml.safe_dump(cfg), encoding="utf-8")
34         return SportsDataCLI(config_path=str(cfg_path))
35
36
37     def _import_args(**kw):
38         base = dict(
39             sport=None, video_path="nonexistent.mp4", annotations_path=None,
40             match_name=None, video_id=None, license="Proprietary", fps=30.0,
resolution="1080p",
41         )
42         base.update(kw)
43         return SimpleNamespace(**base)
44
45
46     def _write_csv(path, header, rows):
47         with open(path, "w", newline="", encoding="utf-8") as f:
48             w = _csv.writer(f)
49             w.writerow(header)
50             for r in rows:
51                 w.writerow(r)
52
53
54     # ----- #
55     # import-local: every sport, both JSON and CSV (covers dispatch + parse) #
56     # ----- #
```

```

57
58     def test_import_local_badminton_json(cli, tmp_path):
59         p = tmp_path / "bad.json"
60         p.write_text(json.dumps([
61             {"rally_number": 1, "start_time": 10.0, "end_time": 20.0, "shots_count": 5,
62             "ending_reason": "winner"},
63             {"rally_number": 2, "start_time": 25.0, "end_time": 30.0, "shots_count": 1,
64             "ending_reason": "service_fault"},
65         ]), encoding="utf-8")
66         cli.import_local(_import_args(sport="badminton", annotations_path=str(p),
67         video_id="bad1", match_name="Bad"))
68         assert len(cli.db.get_badminton_rallies("bad1")) == 2
69
70     def test_import_local_pickleball_csv(cli, tmp_path):
71         p = tmp_path / "pb.csv"
72         _write_csv(p, ["rally_number", "start_time", "end_time", "shots_count",
73         "ending_reason"],
74         [[1, 10.0, 20.0, 6, "winner"], [2, 25.0, 27.0, 1, "service_fault"]])
75         cli.import_local(_import_args(sport="pickleball", annotations_path=str(p),
76         video_id="pb1", match_name="PB"))
77         assert len(cli.db.get_pickleball_rallies("pb1")) == 2
78
79     def test_import_local_tabletennis_json(cli, tmp_path):
80         p = tmp_path / "tt.json"
81         p.write_text(json.dumps([
82             {"rally_number": 1, "start_time": 5.0, "end_time": 9.0, "shots_count": 7,
83             "ending_reason": "winner"},
84         ]), encoding="utf-8")
85         cli.import_local(_import_args(sport="tabletennis", annotations_path=str(p),
86         video_id="tt1", match_name="TT"))
87         assert len(cli.db.get_tabletennis_rallies("tt1")) == 1
88
89     def test_import_local_tennis_csv(cli, tmp_path):
90         p = tmp_path / "tn.csv"
91         _write_csv(p, ["point_number", "start_time", "end_time", "server", "ending_type"],
92         [[1, 10.0, 20.0, "player1", "winner"]])
93         cli.import_local(_import_args(sport="tennis", annotations_path=str(p),
94         video_id="tn1", match_name="TN"))
95         assert len(cli.db.get_tennis_points("tn1")) == 1
96
97     def test_import_local_cricket_json(cli, tmp_path):
98         p = tmp_path / "ck.json"
99         p.write_text(json.dumps([
100             {"over": 0, "ball": 1, "start_time": 5.0, "end_time": 15.0, "runs": 4, "wicket":
101             False},
102         ]), encoding="utf-8")
103         cli.import_local(_import_args(sport="cricket", annotations_path=str(p),
104         video_id="ck1", match_name="CK"))
105         assert len(cli.db.get_cricket_deliveries("ck1")) == 1
106

```

32. assistant (2026-06-07T06:42:09.362Z)

Let me check the README to understand the overall architecture better:

33. user (2026-06-07T06:42:10.289Z)

```

1      # Sports Data Collector & Curator
2
3      A clean, extensible Python codebase and CLI toolchain designed to discover, ingest,
4      validate, and curate sports video datasets. The repository stores annotations and video metadata

```

in an SQLite database, supports sport-specific tables and schemas, manages video storage outside of Git, and enforces strict commercial license compliance.

```
4
5 > **Pipeline role:** this repo is the *data producer* feeding the **badminton-highlight-indexer**, which trains/evaluates an offline rally segmenter on this curated, commercially-licensed data. See [docs/PIPELINE_ROLE.md](docs/PIPELINE_ROLE.md) for why `export`/`fetch` exist and how the two repos fit together.
```

```
6
7 ---
8
9 ## Features
```

```
10
11 - **SQLite Backend**: Centralized `sports_metadata.db` with a generic `videos` table and sport-specific annotation tables (`badminton_rallies`, `tennis_points`, `cricket_deliveries`, `pickleball_rallies`, `tabletennis_rallies`).
```

```
12 - **Commercial License Flagging**: Each ingested source license is checked against a configured whitelist (`MIT`, `Apache-2.0`, `BSD`, `CC0`, `CC-BY`, `Public-Domain`, `Proprietary`) and the resulting `is_commercial` flag is stored alongside the video. Non-commercial sources (like `CC-BY-NC-SA`) are flagged as non-commercial but still ingested ? nothing is discarded automatically. The whitelist is read from `config/config.yaml` consistently across the parser, validator, and CLI.
```

```
13 - **Overlap & Duration Validation**: Ensures chronological event bounds (e.g. rally start/end times) do not overlap and that durations are positive.
```

```
14 - **Stroke-level CSV Parser**: Ingests ShuttleSet/CoachAI-style stroke CSV logs and aggregates them into rally durations using configurable start/end time buffers.
```

```
15 - **Local Ingestion Mode**: Manually registers local videos and associated CSV/JSON annotation files directly into the database as curated entries.
```

```
16 - **Interactive Curation CLI & Visual Play Harness**: Let's you query DB status, list staging entries, review sample timestamps, and approve/reject them. It includes a built-in play function that opens the video in your default browser at the exact rally segment (using HTML5 media fragments `#t=start,end` for local files or YouTube start time markers `&t=start` for remote videos) for manual visual verification.
```

```
17
18 ---
19
20 ## Installation
```

```
21
22 Ensure you have Python 3.9+ installed.
```

```
23
24 1. Clone or download the repository.
25 2. Install the package and dependencies:
```

```
26 ```bash
27 pip install -e .           # core: curate / import-local / export / status
28 pip install -e ".[fetch]"  # adds yt-dlp for video acquisition (fetch / crawler)
29 pip install -e ".[dev]"    # test tooling (pytest, pytest-cov)
30 ```
```

```
31
32 ### External requirements (not pip-installable)
```

```
33
34 The `fetch` command and the ShuttleSet crawler shell out to external binaries.
35 They degrade gracefully (a clear error / skipped download) when missing, but are
36 **required** for video acquisition:
```

```
37
38 - **FFmpeg / ffprobe** ? used to probe the real resolution/fps of downloaded files.
39 - **A JS runtime (e.g. [deno](https://deno.com))** ? needed by yt-dlp to solve
40   YouTube's challenge for high-resolution formats. `fetch` auto-detects a deno
41   install at `~/deno/bin`.
```

```
42
43 Core curation/import/export/status commands work without any of these.
```

```
44
45 ---
46
47 ## Usage
```

```
48
49 All commands should be executed from the root of the project. Make sure `PYTHONPATH` is
```

```

set to the current directory:
50
51     ```bash
52     # On Windows PowerShell:
53     $env:PYTHONPATH="."
54     ```
55
56     ### 1. View Database Status
57     To check the current statistics of staging vs. curated items:
58     ```bash
59     python -m src.cli.curate status
60     ```
61
62     ### 2. Ingest External Datasets to Staging
63     To parse external raw stroke-level datasets (e.g. CoachAI format) and register them in
staging:
64     ```bash
65     python -m src.cli.ingest --sport badminton --parser coachai --file-path
data/staging_coachai_match.csv --license CC-BY-NC-SA --video-id staging_match_non_comm
66     ```
67
68     ### 3. Launch the Interactive Curation Tool & Play Harness
69     To review staging items, inspect their timestamps, play individual rallies in the
browser to visually verify their start and end bounds, and approve or reject them:
70     ```bash
71     python -m src.cli.curate curate
72     ```
73
74     ### 4. Manually Import Local Videos & Annotations
75     To register a local video file alongside its JSON/CSV annotation file (bypasses
staging):
76     ```bash
77     python -m src.cli.curate import-local --sport badminton --video-path
"data/videos/my_custom_match.mp4" --annotations-path "data/mock_local_badminton_anno.json"
--match-name "My Custom Local Match" --license Proprietary
78     ```
79     `import-local` works for every configured sport ? `badminton`, `tennis`, `cricket`,
`pickleball`, and `tabletennis` ? from either a JSON or a CSV annotations file. For the
canonical rally CSV schema and the rater-facing instructions used to produce these files (e.g.
via an annotation vendor), see [docs/annotation/](docs/annotation/).
80
81     ### 5. Fetch / Upgrade Videos to Best Resolution
82     Videos are **always downloaded at the best resolution available** (via yt-dlp `bv*+ba/b
-S res,...`, merged to MP4); the actual width/height/fps is ffprobed and stored ? never assumed.
The `fetch` command (re)downloads videos for entries already in the DB and updates
`local_path`/`resolution`/`fps` **in place**, preserving all rally annotations and curation
status (no clean slate needed):
83     ```bash
84     # Upgrade every sub-720p video to best resolution, using a logged-in browser's cookies:
85     python -m src.cli.curate fetch --cookies-from-browser firefox
86
87     # A single video, forcing a re-download even if a good local copy exists:
88     python -m src.cli.curate fetch --video-id shuttlest_1 --force --cookies-from-browser
firefox
89     ```
90     Flags: `--sport`, `--video-id`, `--status {staging,curated,all}` (default `all`),
`--min-height` (skip videos already at/above it unless `--force`; default `720`), `--max-height`
(cap resolution, e.g. `1080`; default best available), `--force`, `--include-noncommercial`,
`--cookies-from-browser`/`--cookies`, `--player-client`. A global resolution cap can also be set
via `video_storage.max_height` in `config/config.yaml`. Re-runs are cheap and resumable; a
download that falls below `--min-height` is treated as a **failure** (discarded, DB left
unchanged) so a flaky low-res result never overwrites a good entry.
91
92     > **Authenticated cookies are strongly recommended.** Anonymous YouTube downloads are
throttled and non-deterministic (often falling back to 144p). Pass `--cookies-from-browser

```

```

firefox` (or `chrome`/`edge`/`brave`, fully closed) ? or `--cookies cookies.txt` ? to use a
logged-in (ideally Premium) session for fast, reliable best-resolution downloads. Chrome's app-
bound cookie encryption may block direct reads on Windows; Firefox reads cleanly even while
open.
93      >
94      > **JS runtime:** yt-dlp needs one (e.g. [deno](https://deno.com)) to solve YouTube's
challenges for high-res formats; `fetch` auto-detects a deno install at `~/deno/bin`. The
default `--player-client` avoids `web`/`web_safari`, which YouTube forces into SABR streaming
(drops to 144p).
95
96      ### 6. Export Annotations as Indexer-Ready Eval/Training Sets
97      To turn curated annotations into ground-truth files the badminton-highlight-indexer can
score and train against, export per-video `start,end` CSVs plus a `manifest.json`:
98      ```bash
99      # Curated + commercially-safe videos across all supported sports (default):
100     python -m src.cli.curate export --out-dir data/eval_export
101
102     # One sport, also including staging entries:
103     python -m src.cli.curate export --sport badminton --include-staging
104     ```
105     The exporter writes `//.csv` (decimal-second `start,end` rows)
and a `manifest.json` pairing each CSV with its video `local_path`/`video_url`, fps, resolution,
and license. The indexer keys videos by **file MD5**, not by our `video_id`, so the downstream
eval/training run must process the exact video file named in the manifest. Flags: `--sport`
(restrict to one sport), `--include-staging` (default: curated only), `--include-noncommercial`
(default: commercial-safe only).
106
107     ---
108
109     ## Configuration
110
111     Adjust settings in `config/config.yaml`:
112     - **`database_path`**: Path to the SQLite database.
113     - **`video_storage`**: Local folder path and automatic download size limit.
114     - **`commercial_licenses`**: Whitelist of licenses allowed for commercial use.
115     - **`supported_sports`**: Allowed sports categories.
116
117     ---
118
119     ## Documentation
120
121     - [docs/PIPELINE_ROLE.md](docs/PIPELINE_ROLE.md) ? how this repo feeds the **badminton-
highlight-indexer** (`export` / `fetch`).
122     - [docs/DECISIONS.md](docs/DECISIONS.md) ? collector-local decision records
(**ADR-001**: keep explicit per-sport code instead of a registry, with an "adding a new sport"
checklist).
123     - [docs/annotation/](docs/annotation/) ? the vendor annotation package used to produce
golden rally labels:
124         - [ROORA_BRIEF.md](docs/annotation/ROORA_BRIEF.md) ? vendor brief + the canonical
rally CSV schema.
125         - [ANNOTATION_GUIDE.md](docs/annotation/ANNOTATION_GUIDE.md) ? the illustrated rater
guide (singles & doubles).
126         - [INTAKE_MANIFEST_TEMPLATE.csv](docs/annotation/INTAKE_MANIFEST_TEMPLATE.csv) ? per-
video intake sheet.
127         - [NDA_TEMPLATE.md](docs/annotation/NDA_TEMPLATE.md) ? mutual NDA template (have
counsel review).
128
129     ---
130
131     ## Running Tests
132
133     We use `pytest` for unit and integration testing:
134     ```bash
135     python -m pytest tests/
136     python -m pytest tests/ --cov=src --cov-report=term-missing # with coverage (~87%)

```

137 ` ``
138

34. assistant (2026-06-07T06:42:12.720Z)

Let me look at the directory structure to see where outputs currently go:

35. user (2026-06-07T06:42:13.211Z)

```
data/eval_export/badminton/my_custom_local_match.csv
data/eval_export/badminton/shuttleaset_1.csv
data/eval_export/badminton/shuttleaset_10.csv
data/eval_export/badminton/shuttleaset_11.csv
data/eval_export/badminton/shuttleaset_12.csv
data/eval_export/badminton/shuttleaset_13.csv
data/eval_export/badminton/shuttleaset_14.csv
data/eval_export/badminton/shuttleaset_15.csv
data/eval_export/badminton/shuttleaset_16.csv
data/eval_export/badminton/shuttleaset_17.csv
data/eval_export/badminton/shuttleaset_18.csv
data/eval_export/badminton/shuttleaset_19.csv
data/eval_export/badminton/shuttleaset_2.csv
data/eval_export/badminton/shuttleaset_20.csv
data/eval_export/badminton/shuttleaset_21.csv
data/eval_export/badminton/shuttleaset_22.csv
data/eval_export/badminton/shuttleaset_23.csv
data/eval_export/badminton/shuttleaset_24.csv
data/eval_export/badminton/shuttleaset_25.csv
data/eval_export/badminton/shuttleaset_3.csv
```

36. assistant (2026-06-07T06:42:14.358Z)

Let me check the validation directory:

37. user (2026-06-07T06:42:14.709Z)

```
total 20
drwxr-xr-x 1 avidu 197609   0 Jun  6 15:46 .
drwxr-xr-x 1 avidu 197609   0 Jun  6 15:46 ..
-rw-r--r-- 1 avidu 197609 1198 Jun  6 15:46 pilot_badminton_01_report.json
-rw-r--r-- 1 avidu 197609  716 Jun  6 15:46 pilot_badminton_01_report.md
-rw-r--r-- 1 avidu 197609 1327 Jun  6 15:46 pilot_badminton_01_review.csv
```

38. assistant (2026-06-07T06:42:16.356Z)

Good! Let me look at an example report to understand the current output format:

39. user (2026-06-07T06:42:16.485Z)

```
1      # Delivery validation ? pilot_badminton_01
2
3      - rallies: **24**
4      - issues: **4** (blocker 1, high 0, medium 3, low 0)
5      - structurally-clean rallies: **21/24** (87%)
6      - gate: **BLOCKED**
7
8      ## BLOCKER
9      - (overlap) Rally 1 (ends 21.02s) overlaps Rally 2 (starts 2.50s) by 18.52s ? rallies
must be disjoint.
10
11     ## MEDIUM
12     - (shot_density) Rally 2: 2 shots over 30.0s (0.07/s) is implausibly low ? the span may
be inflated by a stray box.
13     - (shot_density) Rally 7: 2 shots over 16.0s (0.12/s) is implausibly low ? the span may
```


be inflated by a stray box.
14 - (shot_density) Rally 16: 2 shots over 17.5s (0.11/s) is implausibly low ? the span may
be inflated by a stray box.
15
16 ## Rallies to review: [2, 7, 16]

40. assistant (2026-06-07T06:42:17.728Z)

Let me also check an example json report:

41. user (2026-06-07T06:42:17.854Z)

```
1  {
2    "source": "pilot_badminton_01",
3    "fps": 59.94,
4    "n_rallies": 24,
5    "n_issues": 4,
6    "counts_by_severity": {
7      "blocker": 1,
8      "high": 0,
9      "medium": 3,
10     "low": 0
11   },
12   "n_blockers": 1,
13   "passed": false,
14   "flagged_rallies": [
15     2,
16     7,
17     16
18   ],
19   "issues": [
20     {
21       "severity": "medium",
22       "code": "shot_density",
23       "rally_number": 2,
24       "message": "Rally 2: 2 shots over 30.0s (0.07/s) is implausibly low \u2014 the
span may be inflated by a stray box."
25     },
26     {
27       "severity": "blocker",
28       "code": "overlap",
29       "rally_number": 2,
30       "message": "Rally 1 (ends 21.02s) overlaps Rally 2 (starts 2.50s) by 18.52s \u2014
rallies must be disjoint."
31     },
32     {
33       "severity": "medium",
34       "code": "shot_density",
35       "rally_number": 7,
36       "message": "Rally 7: 2 shots over 16.0s (0.12/s) is implausibly low \u2014 the
span may be inflated by a stray box."
37     },
38     {
39       "severity": "medium",
40       "code": "shot_density",
41       "rally_number": 16,
42       "message": "Rally 16: 2 shots over 17.5s (0.11/s) is implausibly low \u2014 the
span may be inflated by a stray box."
43     }
44   ]
45 }
```

42. assistant (2026-06-07T06:42:19.205Z)

Now let me check if there's a CVAT sample file to understand the format better:

43. user (2026-06-07T06:42:19.568Z)

(Bash completed with no output)

44. assistant (2026-06-07T06:42:20.904Z)

Let me check what tests exist for the CVAT parser and validator:

45. user (2026-06-07T06:42:21.024Z)

```
1      """Tests for the CVAT rally adapter (src/parsers/cvat/rally_cvat.py).
2
3      The adapter's whole reason for existing is to (a) recompute rally time from the
4      box FRAME RANGE at the true fps ? ignoring the vendor's wrong start/end_time
5      attributes ? and (b) report content defects rather than silently fixing them.
6      These tests pin both behaviours, plus the end-to-end hand-off to import-local.
7      """
8      import csv as _csv
9
10     from src.parsers.cvat.rally_cvat import (
11         RallyRecord, parse_cvat_boxes, build_records, detect_issues, convert,
12         write_canonical_csv, load_canonical_csv, build_summary, render_validation_md,
13         write_review_csv,
14     )
15     from src.cli.curate import SportsDataCLI
16
17
18     # --- helpers -----
19
20     def _img_box(frame, attrs):
21         rows = "".join(f'<attribute name="{k}">{v}</attribute>' for k, v in attrs.items())
22         return (f'<image id="{frame}" name="frame_{frame:06d}" width="1920" height="1080">'
23             f'<box label="rally" xtl="1" ytl="1" xbr="2" ybr="2">{rows}</box></image>')
24
25
26     def _write_cvat(path, images, step=30, stop_frame=1000):
27         body = "".join(images)
28         xml = (
29             '<?xml version="1.0" encoding="utf-8"?><annotations><version>1.1</version>'
30             f'<meta><job><stop_frame>{stop_frame}</stop_frame>'
31             f'<frame_filter>step={step}</frame_filter></job></meta>'
32             f'{body}</annotations>'
33         )
34         path.write_text(xml, encoding="utf-8")
35         return str(path)
36
37
38     def _rec(rn, start, end, shots=5, reason="winner", fmin=None, fmax=None, n=3,
39 outcome=None):
40         return RallyRecord(
41             rally_number=rn, start_time=start, end_time=end, shots_count=shots,
42             ending_reason=reason, last_shot_outcome=outcome,
43             score_before=None, score_after=None, notes=None,
44             frame_min=fmin if fmin is not None else int(start * 60),
45             frame_max=fmax if fmax is not None else int(end * 60), n_boxes=n)
46
47     # --- the core fix: time from frames, vendor times ignored -----
48
49     def test_time_recomputed_from_frames_not_attributes(tmp_path):
50         # rally 1 spans source frames 60..120; the vendor's start/end attributes are
51         # deliberately wrong (the frame/900 bug). At 60 fps the truth is 1.0..2.0s.
```

```

52     xml = _write_cvat(tmp_path / "j.xml", [
53         _img_box(60, {"rally_number": "1.0", "start_time": "0.2", "end_time": "1.4",
54             "shots_count": "6.0", "ending_reason": "winner",
55             "score_before (A-B)": "0-0", "notes": "clean"}),
56         _img_box(90, {"rally_number": "1.0", "start_time": "0.2", "end_time": "1.4"}),
57         _img_box(120, {"rally_number": "1.0", "start_time": "0.2", "end_time": "1.4"}),
58     ])
59     res = convert(xml, fps=60.0)
60     assert len(res.records) == 1
61     r = res.records[0]
62     assert r.start_time == 1.0 and r.end_time == 2.0      # frames/fps, not 0.2/1.4
63     assert r.vendor_start_time == "0.2"                  # preserved for transparency
64     assert r.shots_count == 6
65     assert r.ending_reason == "winner"
66     assert r.score_before == "0-0"                        # "(A-B)" alias resolved
67     assert r.notes == "clean"
68
69
70 def test_step_and_fps_read_from_meta_and_arg(tmp_path):
71     xml = _write_cvat(tmp_path / "j.xml", [
72         _img_box(0, {"rally_number": "1", "shots_count": "3"}),
73         _img_box(30, {"rally_number": "1"}),
74     ], step=30)
75     res = convert(xml, fps=60.0)
76     assert res.step == 30
77     assert res.fps == 60.0
78
79
80 def test_fps_required_when_unavailable(tmp_path):
81     xml = _write_cvat(tmp_path / "j.xml", [_img_box(0, {"rally_number": "1"})])
82     try:
83         convert(xml) # no fps, no video_path
84     except ValueError as e:
85         assert "fps" in str(e).lower()
86     else:
87         raise AssertionError("expected ValueError when fps cannot be determined")
88
89
90 # --- track-mode parsing + outside filtering -----
91
92 def test_track_mode_and_outside_filtering(tmp_path):
93     xml = (
94         '<?xml version="1.0"?><annotations><meta><job>'
95         '<stop_frame>1000</stop_frame><frame_filter>step=30</frame_filter></job></meta>'
96         '<track id="0" label="rally">'
97         '<box frame="60" outside="0" xtl="1" ytl="1" xbr="2" ybr="2">'
98         '<attribute name="rally_number">1</attribute>'
99         '<attribute name="shots_count">4</attribute></box>'
100         '<box frame="120" outside="0" xtl="1" ytl="1" xbr="2" ybr="2">'
101         '<attribute name="rally_number">1</attribute></box>'
102         '<box frame="150" outside="1" xtl="1" ytl="1" xbr="2" ybr="2">' # terminator
103         '<attribute name="rally_number">1</attribute></box>'
104         '</track></annotations>'
105     )
106     p = tmp_path / "track.xml"
107     p.write_text(xml, encoding="utf-8")
108     meta, boxes = parse_cvat_boxes(str(p))
109     assert len(boxes) == 2      # the outside=1 terminator dropped
110     recs = build_records(boxes, fps=60.0)
111     assert recs[0].frame_max == 120      # not 150 (the terminator)
112     assert recs[0].end_time == 2.0
113
114
115 # --- issue detection: surfaced, not silently fixed -----
116

```

```

117     def test_overlap_is_a_blocker():
118         recs = [_rec(1, 1.0, 5.0), _rec(2, 4.0, 8.0)] # r2 starts before r1 ends
119         issues = detect_issues(recs)
120         overlaps = [i for i in issues if i.code == "overlap"]
121         assert len(overlaps) == 1 and overlaps[0].severity == "blocker"
122
123
124     def test_zero_duration_is_a_blocker():
125         recs = [_rec(1, 2.0, 2.0, fmin=120, fmax=120, n=1)]
126         issues = detect_issues(recs)
127         assert any(i.code == "zero_duration" and i.severity == "blocker" for i in issues)
128
129
130     def test_offvocab_ending_reason_flagged():
131         recs = [_rec(1, 1.0, 3.0, reason="into_net")]
132         issues = detect_issues(recs)
133         assert any(i.code == "ending_reason_vocab" for i in issues)
134         # the adopted forced/unforced terms are accepted, not flagged
135         ok = detect_issues([_rec(1, 1.0, 3.0, reason="forced_error")])
136         assert not any(i.code == "ending_reason_vocab" for i in ok)
137
138
139     def test_low_shot_density_flags_inflated_span():
140         recs = [_rec(1, 0.0, 16.0, shots=2)] # 2 shots over 16s -> 0.12/s
141         issues = detect_issues(recs)
142         assert any(i.code == "shot_density" for i in issues)
143
144
145     def test_outcome_vocab_flagged():
146         bad = detect_issues([_rec(1, 1.0, 3.0, outcome="long")])
147         assert any(i.code == "outcome_vocab" for i in bad)
148         # the agreed outcomes are accepted, not flagged
149         ok = detect_issues([_rec(1, 1.0, 3.0, outcome="out")])
150         assert not any(i.code == "outcome_vocab" for i in ok)

```

46. assistant (2026-06-07T06:42:22.633Z)

Perfect! Now I have a comprehensive understanding of the codebase. Let me create a detailed summary document based on my exploration:

47. user (2026-06-07T06:42:23.146Z)

Chapter marked: "Repository Exploration Complete". Continue your current work.

48. assistant (2026-06-07T06:43:26.608Z)

Now I'll provide you with a comprehensive map of the sports-data-collector repository:

COMPREHENSIVE REPOSITORY MAP: SPORTS-DATA-COLLECTOR

1. CLI ENTRYPOINTS

****File:**** `C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py` (lines 805-921)

The `SportsDataCLI` class exposes these subcommands via argparse:

Subcommand	Handler	Lines	Purpose
`status`	`cli.status()`	38-50, 902	Display summary of videos in DB (count by sport/status)
`curate`	`cli.curate()`	619-722, 904-905	Interactive review loop for staging videos; approve/reject/play segments

```
| `import-local` | `cli.import_local(args)` | 52-142, 906-907 | Manually register local video +
JSON/CSV annotations into DB (any sport) |
| `convert-cvat` | `cli.convert_cvat(args)` | 526-560, 908-909 | Convert vendor CVAT XML export
to canonical per-rally CSV + issues report |
| `validate-delivery` | `cli.validate_delivery(args)` | 562-617, 910-911 | Full validation suite
on CVAT/CSV; emit `_report.{md,json}` + `_review.csv` |
| `fetch` | `cli.fetch(args)` | 330-426, 914-915 | Download/re-download videos at best
resolution; update DB in place |
| `export` | `cli.export_eval_set(args)` | 428-524, 912-913 | Export curated annotations as
indexer-ready eval/training CSVs + manifest.json |
```

2. PER-VIDEO PROCESSING PIPELINE

****Ingestion Path 1: `import-local` (Direct Local File)****

****Entry:**** `src/cli/curate.py:52-142`

1. ****Parse annotations**** ? `\_parse\_local\_annotations()` (lines 144-290)
 - Reads `.json` or `.csv` files
 - Dispatches per-sport: BadmintonRally, TennisPoint, CricketDelivery, PickleballRally, TableTennisRally
 - Uses `safe\_int()` / `safe\_float\_required()` from `src/core/parsing.py:9-29` for robust casting
2. ****Validate**** ? `DatasetValidator.validate\_{sport}\_rallies/points/deliveries()` (src/core/validator.py:29-144)
 - Checks: chronological order, no overlaps, positive duration
 - Returns `(is\_valid: bool, errors: List[str])`
3. ****Create VideoMetadata**** ? `VideoMetadata(...)` (lines 109-121)
 - Sets `status = CurationStatus.CURATED` (bypasses staging)
 - Stores: video_id, sport, local_path, license_type, fps, resolution
4. ****Insert to DB**** ? `SportsDatabase.insert\_video()` + sport-specific insert (lines 124-136)
 - `insert\_badminton\_rallies()`, `insert\_tennis\_points()`, etc.
 - Uses UPSERT logic to preserve child rows when updating

****Ingestion Path 2: `convert-cvat` (Vendor CVAT Export ? CSV)****

****Entry:**** `src/cli/curate.py:526-560`

****Key Pipeline (src/parsers/cvat/rally_cvat.py):****

1. ****Parse CVAT XML**** ? `parse\_cvat\_boxes(xml\_path)` (lines 140-193)
 - Extracts `` (frame-mode) or `` (interpolation-mode) boxes
 - Returns `(meta: {step, stop\_frame}, boxes: [{frame, attrs}])`
2. ****Recompute time from frames**** ? `build\_records(boxes, fps)` (lines 205-239)
 - ****Critical:**** derives `start\_time = frame\_min / fps`, `end\_time = frame\_max / fps`
 - ****Ignores**** vendor's `start\_time`/`end\_time` attributes (they're often wrong due to step/fps mismatch)
 - Resolves CVAT attribute name aliases (e.g., "score_before (a-b)" ? "score_before")
3. ****Detect issues**** ? `detect\_issues(records, stop\_frame, ...)` (lines 242-322)
 - ****Blocker issues**** (halt import): overlap, zero_duration
 - ****High/Medium/Low issues**** (flag for review): ending_reason_vocab, shot_density, possible_missed_rally, outcome_vocab, frame_out_of_range
 - ****Does NOT flag**** sampling density; downstream padding absorbs it
4. ****Write canonical CSV**** ? `write\_canonical\_csv(records, out\_path)` (lines 356-372)
 - Columns: `rally\_number`, `start\_mmss`, `end\_mmss`, `start\_time`, `end\_time`, `shots\_count`,

```

`ending_reason`, `last_shot_outcome`, `score_before`, `score_after`, `notes`
- Output ready for `import-local`

5. **Emit issues report** ? `render_issue_report(result, source)` (lines 375-396)
- Human-readable text grouped by severity
- Also written to `--issues` file if specified

---

#### **Ingestion Path 3: `validate-delivery` (Validation + Structured Reports)**

**Entry:** `src/cli/curate.py:562-617`

1. **Load from CVAT or CSV:**
- CVAT: calls `convert(...)` ? records + issues
- CSV: calls `load_canonical_csv()` (lines 424-456) ? records + calls `detect_issues()`

2. **Build summary (JSON)** ? `build_summary(records, issues, ...)` (lines 459-476)
- Returns dict with: `n_rallies`, `n_issues`, `counts_by_severity`, `n_blockers`, `passed`,
`flagged_rallies`, `issues` (detailed array)

3. **Render validation report (MD)** ? `render_validation_md(records, issues, ...)` (lines
479-509)
- Human scoreboard + issues by severity + list of flagged rally numbers

4. **Generate review sheet (CSV)** ? `write_review_csv(records, issues, out_path)` (lines
512-534)
- Annotatable spreadsheet with `auto_flags` column + blank `true_*` / `reviewer_comment` cols
for human review

5. **Output files** (lines 600-610):
- `{vid}_report.json` ? machine summary
- `{vid}_report.md` ? human report
- `{vid}_review.csv` ? annotation sheet

**Exit code:** 0 if no blockers; 2 if blockers exist (gates pipeline)

---

#### **Fetch Path: `fetch` (Download or Re-download Videos)**

**Entry:** `src/cli/curate.py:330-426`

1. **Collect target videos** by status/sport (lines 356-369)
2. **For each video:**
- Skip if no URL or already-good local file (unless `--force`)
- Call `_fetch_one_with_retry()` (lines 292-328) ? `download_best_video()` from
`src/core/downloader.py`
- Probe resolution/fps ? `probe_resolution_fps()` from `src/core/downloader.py`
3. **Update DB in place** (line 407) via UPSERT (preserves rally annotations, status)
4. **Emit fetch summary table** (lines 416-426)

---

#### **Export Path: `export` (Evaluation/Training Sets)**

**Entry:** `src/cli/curate.py:428-524`

1. **For each video/sport:**
- Check commercial flag + status (default: CURATED + commercial-safe)
- Call `get_segment_intervals()` ? returns `[(start_time, end_time), ...]` (lines 527-546)
- Write `**Write manifest.json** (lines 500-509)
- Per-video entry: `video_id`, `sport`, `csv` (relative path), `local_path`, `video_url`,
`fps`, `resolution`, `license_type`, `is_commercial`, `status`, `num_segments`

```

```

- Metadata: `commercial_only`, `statuses`, `total_videos`, `total_segments`

---

3. DATA MODEL / STORE

#### **Database Schema** (src/core/db.py:52-166)

**Location:** `data/sports_metadata.db` (configurable via `config/config.yaml:database_path`)

**Core Table: `videos`** (lines 57-71)
```sql
CREATE TABLE videos (
 video_id TEXT PRIMARY KEY,
 sport TEXT NOT NULL,
 'badminton'|'tennis'|'cricket'|'pickleball'|'tabletennis'
 video_url TEXT,
 local_path TEXT,
 license_type TEXT NOT NULL,
 license_url TEXT,
 is_commercial INTEGER NOT NULL,
 status TEXT NOT NULL DEFAULT 'staging',
 match_name TEXT NOT NULL,
 fps REAL,
 resolution TEXT
)
```

**Sport-Specific Tables** (all with `ON DELETE CASCADE`):

- **`badminton_rallies`** (lines 74-88): PK(video_id, rally_number)
  - Columns: `rally_number` INT, `start_time` REAL, `end_time` REAL, `score_before` TEXT, `score_after` TEXT, `shots_count` INT, `ending_reason` TEXT, `last_shot_outcome` TEXT

- **`tennis_points`** (lines 92-104): PK(video_id, point_number)
  - Columns: `point_number` INT, `start_time` REAL, `end_time` REAL, `set_score` TEXT, `game_score` TEXT, `server` TEXT, `ending_type` TEXT

- **`cricket_deliveries`** (lines 108-122): PK(video_id, over_num, ball_num)
  - Columns: `over_num` INT, `ball_num` INT, `start_time` REAL, `end_time` REAL, `batsman` TEXT, `bowler` TEXT, `runs` INT, `wicket` INT, `extras` INT

- **`pickleball_rallies`** (lines 126-139): PK(video_id, rally_number)
  - Same as badminton (rally-based sport)

- **`tabletennis_rallies`** (lines 143-156): PK(video_id, rally_number)
  - Same as badminton (rally-based sport)

**Generic Reader** (lines 16-22, 527-546):
```python
SEGMENT_TABLES = {
 SportType.BADMINTON: ("badminton_rallies", "rally_number"),
 SportType.TENNIS: ("tennis_points", "point_number"),
 SportType.CRICKET: ("cricket_deliveries", "over_num", "ball_num"),
 SportType.PICKLEBALL: ("pickleball_rallies", "rally_number"),
 SportType.TABLE_TENNIS: ("tabletennis_rallies", "rally_number"),
}
```

---

#### **Canonical Data Shapes** (src/core/schemas/)

**VideoMetadata** (src/core/schemas/base.py:29-40)

```

```

```python
class VideoMetadata(BaseModel):
 video_id: str # unique identifier
 sport: SportType # enum:
badminton|tennis|cricket|pickleball|tabletennis
 video_url: Optional[str] # remote source
 local_path: Optional[str] # local file
 license_type: LicenseType # enum: MIT|Apache-2.0|CC-BY|CC-BY-
NC|Proprietary|Unknown...
 license_url: Optional[str]
 is_commercial: bool # license whitelist match
 status: CurationStatus # enum: staging|curated|rejected
 match_name: str # human name
 fps: Optional[float] # video metadata
 resolution: Optional[str] # video metadata (e.g., '1080p')
...

```

```

BadmintonRally (src/core/schemas/badminton.py:4-21)
```python
class BadmintonRally(BaseModel):
    video_id: str
    rally_number: int # chronological index
    start_time: float # seconds (must be >= 0, < end_time)
    end_time: float
    score_before: Optional[str] # e.g., '0-0'
    score_after: Optional[str] # e.g., '1-0'
    shots_count: Optional[int] # stroke count
    ending_reason: Optional[str] #
'winner'|'forced_error'|'unforced_error'|'service_fault'|'let'|'other'
    last_shot_outcome: Optional[str] # 'in'|'net'|'out'|'n_a'
...

```

****TennisPoint, CricketDelivery, PickleballRally, TableTennisRally** follow similar patterns.**

4. EXISTING OUTPUT ARTIFACTS

A. Export CSVs (curate.py:478-482)

****Location:**** `

****Format:**** 2-column CSV (decimal seconds)

...

start,end

10.5,20.3

25.1,30.8

...

****Producer:**** `export\_eval\_set()` calls `get\_segment\_intervals()` (db.py:527-546)

B. Manifest JSON (curate.py:500-509)

****Location:**** `

****Shape:****

```json

```

{
 "commercial_only": true,
 "statuses": ["curated"],
 "total_videos": 50,
 "total_segments": 1200,
 "eval_sets": [

```



```

 {
 "video_id": "shuttleset_1",
 "sport": "badminton",
 "csv": "badminton/shuttleset_1.csv",
 "local_path": "/path/to/video.mp4",
 "video_url": "https://youtube.com/...",
 "match_name": "...",
 "fps": 60.0,
 "resolution": "1080p",
 "license_type": "Proprietary",
 "is_commercial": true,
 "status": "curated",
 "num_segments": 24
 }
]
}
...

Producer: `export_eval_set()` (curate.py:500-509)

C. Validation Reports (validate-delivery command)

Location: `

```

## Delivery validation ? <source>

```

- rallies: **24**
- issues: **4** (blocker 1, high 0, medium 3, low 0)
- structurally-clean rallies: **21/24** (87%)
- gate: **BLOCKED** | **PASS**

```

### BLOCKER

```
- (overlap) Rally 1 (ends 21.02s) overlaps Rally 2 (starts 2.50s) by 18.52s...
```

### MEDIUM

```
- (shot_density) Rally 2: 2 shots over 30.0s (0.07/s) is implausibly low...
```

### Rallies to review: [2, 7, 16]

```

...

`{vid}_report.json` (rally_cvat.py:459-476)
```json
{
    "source": "pilot_badminton_01",
    "fps": 59.94,
    "n_rallies": 24,
    "n_issues": 4,
    "counts_by_severity": {"blocker": 1, "high": 0, "medium": 3, "low": 0},
    "n_blockers": 1,
    "passed": false,
    "flagged_rallies": [2, 7, 16],
    "issues": [
        {
            "severity": "medium",
            "code": "shot_density",
            "rally_number": 2,
            "message": "Rally 2: 2 shots over 30.0s (0.07/s)..."
        }
    ]
}
}

```

```

...

**`{vid}_review.csv`** (rally_cvat.py:512-534)
...

rally_number,start_mmss,end_mmss,dur_s,shots_count,ending_reason,last_shot_outcome,auto_flags,VE
RIFIED(Y/N),true_start_mmss,true_end_mmss,true_ending_reason,true_last_shot_outcome,reviewer_com
ment
1,0:00.000,0:05.000,5.0,5,winner,in,,,,,
2,0:02.500,0:32.500,30.0,2,n/a,n/a,shot_density,,,,,
...

**Producers:** `build_summary()`, `render_validation_md()`, `write_review_csv()`

---

#### **D. CVAT Conversion Issues Report** (convert-cvat command)

**Location:** `--issues` file (optional; also printed to stdout)

**Format:** Human-readable text from `render_issue_report()` (rally_cvat.py:375-396)
...
CVAT rally conversion report ? input.xml
  fps used      : 59.94
  sampling step  : 30 frames (~0.50s)
  rallies parsed : 24
  issues        : 4 (1 blocker)

[BLOCKER]
- (overlap) Rally 1 (ends 21.02s) overlaps Rally 2...

[MEDIUM]
- (shot_density) Rally 2: 2 shots over 30.0s...
...

**Producer:** `render_issue_report()` (rally_cvat.py:375-396)

---

5. VALIDATION: `validate-delivery` IMPLEMENTATION

**Entry Point:** `src/cli/curate.py:562-617`

**Validation Logic:** `src/parsers/cvat/rally_cvat.py:242-322`

**Key Checks:**

| Issue Code | Severity | Condition | Blocker? |
|---|---|---|---|
| `zero_duration` | blocker | `end_time <= start_time` | YES |
| `overlap` | blocker | Rally i starts before Rally i-1 ends | YES |
| `ending_reason_vocab` | medium | `ending_reason` not in {winner, forced_error, unforced_error, service_fault, let, other} | NO |
| `outcome_vocab` | low | `last_shot_outcome` not in {in, net, out, n_a} | NO |
| `shot_density` | medium | `shots_count/duration < 0.3` (for duration >= 5s) ? inflated span | NO |
| `possible_missed_rally` | medium | Gap > 15s between rallies ? check for unlabeled segment | NO |
| `frame_out_of_range` | high | `frame_max > stop_frame` | NO |

**Exit behavior:**
- `validate-delivery` exits with code **2** if any blockers exist (gates pipeline)
- Code **0** otherwise (PASS gate)

---

```

6. CANONICAL DATA SHAPES & FPS/TIMESTAMP HANDLING

Time Representation:

```
- **Primary:** Decimal seconds (float) ? stored in DB and used in all calculations
- **mm:ss format:** Human-readable output via `_mmss(seconds)` (rally_cvat.py:126-130)
  ```python
 def _mmss(seconds: float) -> str:
 m = int(seconds // 60)
 s = seconds - m * 60
 return f"{m}:{s:06.3f}" # e.g., "0:02.500"
  ```
```

Frame ? Seconds Conversion:

```
- **Frame ? seconds:** `seconds = frame_number / fps`
- **Example:** Frame 120 at 60 fps ? 120/60 = 2.0 seconds
- **Critical in CVAT adapter:** Always use **video's true fps** (probed from ffprobe), never
  guess 30 fps
  ```python
 # From rally_cvat.py:325-354
 fps = fps_arg or ffprobe(video_path) or raise ValueError(...)
  ```
```

Sport Profile Abstraction:

The `SEGMENT\_TABLES` mapping (db.py:16-22) allows generic iteration over any sport:

```
```python
SEGMENT_TABLES = {
 SportType.BADMINTON: ("badminton_rallies", "rally_number"),
 SportType.TENNIS: ("tennis_points", "point_number"),
 # ...
}
```

### Usage (db.py:527-546):

```
table, order_by = SEGMENT_TABLES[sport]
cursor.execute(f"SELECT start_time, end_time FROM {table} WHERE video_id = ? ORDER BY
{order_by}", ...)
```

---

## 7. TESTS

\*\*Location:\*\* `C:\Users\avidu\Projects\sports-data-collector\tests\`

```
| Test File | Coverage | Key Tests |
|---|---|---|
| `test_cli.py` | CLI commands (import-local, export, fetch, curate) | All sports (badminton,
tennis, cricket, pickleball, tabletennis); JSON/CSV parsing |
| `test_cvat_parser.py` | CVAT adapter (frame?time, issue detection) | Time recomputation,
overlap/zero-duration blockers, vocabulary checks, output serialization |
| `test_db.py` | Database insert/query | UPSERT logic, cascade delete, per-sport queries |
| `test_validator.py` | DatasetValidator | Overlap detection, duration validation per sport |
| `test_local_import.py` | Local ingestion pipeline | JSON/CSV ? BadmintonRally ? DB |
| `test_parser.py` | Generic parser utilities | ? |
| `test_parsing.py` | `safe_int()`, `safe_float_required()`, `parse_time_string()` | Robust
casting from mixed types |
| `test_schemas.py` | Pydantic models | BadmintonRally validation (start < end, non-negative) |
| `test_downloader.py` | Video download (yt-dlp wrapper) | ? |
| `test_config.py` | Configuration loader | ? |
```

---

## 8. DESIGN RECOMMENDATIONS FOR NEW "RICH OBSERVABILITY REPORT" FEATURE

Based on the architecture, here's where/how to integrate a per-video observability report:

#### **\*\*Ideal Placement:\*\***

1. **\*\*Hook into `export\_eval\_set()` pipeline\*\*** (curate.py:428-524)
  - Called after all videos have been collected and CSVs written
  - Can iterate all exported videos, aggregate metrics, emit final report
  - Write to `/observability\_report.{json,md}`
2. **\*\*Alternative: Standalone command `export-with-observability`\*\***
  - Wrap `export\_eval\_set()` and add observability layer
  - Or create a separate `--emit-observability` flag to `export` command

#### **\*\*What to Observe (candidate metrics):\*\***

- **\*\*Per-video level:\*\***
  - Segment count, total annotated duration, density (segments/hour)
  - FPS distribution, resolution distribution
  - License/commercial breakdown
  - Status distribution (staging vs. curated)
- **\*\*Per-sport level:\*\***
  - Total videos, total segments, coverage stats
  - Segment duration stats (min, max, mean, median)
- **\*\*Cross-cutting:\*\***
  - Data quality scoreboard (validation pass rate from `validate-delivery`)
  - Temporal coverage (do time ranges overlap? gaps?)
  - Download success/failure rates (from `fetch` command)

#### **\*\*Output Format (consistent with existing reports):\*\***

**\*\*`observability\_report.json`\*\*** (machine-readable for dashboards)

```
```json
{
  "timestamp": "2026-06-07T...",
  "total_videos": 50,
  "total_segments": 1200,
  "by_sport": {
    "badminton": {
      "count": 30,
      "segments": 800,
      "avg_duration_s": 4.5,
      "fps_distribution": {"60": 25, "30": 5}
    }
  },
  "quality": {
    "validation_passed": 48,
    "validation_blocked": 2
  }
}
```
```

**\*\*`observability\_report.md`\*\*** (human-readable summary)

```markdown

Data Export Observability Report

Summary

- Total videos exported: 50
- Total segments: 1200
- Coverage: 5400 seconds of annotation

By Sport

| Sport | Videos | Segments | Avg Duration |

...

Quality Metrics

- Validation pass rate: 96% (48/50)

...

Integration with Existing Patterns:

- **Report generation:** Mirror `build_summary()` + `render_validation_md()` from `rally_cvat.py`
- **File writing:** Use same patterns as `validate-delivery` (lines 606-610 in `curate.py`)
- **Data aggregation:** Pull from `manifest.json` entries already built (lines 484-497)
- **Test structure:** Add `test_observability_report.py` alongside `test_cvat_parser.py`

Key Files to Modify:

1. **`src/cli/curate.py`**
 - Add `_emit_observability_report(manifest, out_dir)` method
 - Call from `export_eval_set()` after manifest is finalized (post-line 509)
2. **`src/core/observability.py`** (NEW)
 - `build_observability_summary(manifest: List[Dict]) -> Dict` (analogous to `build_summary()`)
 - `render_observability_md(summary: Dict) -> str` (analogous to `render_validation_md()`)
 - `write_observability_report(summary: Dict, out_dir: str)` (analogous to `validate-delivery` output logic)
3. **`tests/test_observability.py`** (NEW)
 - Test aggregation logic, format consistency
 - Test with various manifest shapes (empty, single sport, multi-sport, mixed statuses)

SUMMARY: WHERE THE PIPELINE WRITES OUTPUT

| Output | Command | Location | Producer Function | Format | Lines |
|-------------------------|--------------------------------|---|--------------------------------------|--|------------------------------------|
| Per-video eval CSVs | <code>export</code> | <code><out_dir>/<sport>/<video_id>.csv</code> | <code>export_eval_set()</code> | start,end (decimal seconds) | <code>curate.py:478-482</code> |
| Manifest | <code>export</code> | <code><out_dir>/manifest.json</code> | <code>export_eval_set()</code> | JSON: video metadata + segment count | <code>curate.py:500-509</code> |
| Validation MD report | <code>validate-delivery</code> | <code><out_dir>/<id>_report.md</code> | <code>render_validation_md()</code> | Human-readable scoreboard + issues | <code>rally_cvat.py:479-509</code> |
| Validation JSON summary | <code>validate-delivery</code> | <code><out_dir>/<id>_report.json</code> | <code>build_summary()</code> | Machine-readable: severity counts, blockers, flagged rallies | <code>rally_cvat.py:459-476</code> |
| Review sheet | <code>validate-delivery</code> | <code><out_dir>/<id>_review.csv</code> | <code>write_review_csv()</code> | Annotatable CSV with auto-flags + blank true_* columns | <code>rally_cvat.py:512-534</code> |
| CVAT issues report | <code>convert-cvat</code> | <code>--issues</code> file (optional) | <code>render_issue_report()</code> | Text: grouped by severity | <code>rally_cvat.py:375-396</code> |
| Canonical rally CSV | <code>convert-cvat</code> | <code>--out</code> file | <code>write_canonical_csv()</code> | CSV: rally_number, start/end times, metadata | <code>rally_cvat.py:356-372</code> |
| Database | All ingestion | <code>data/sports_metadata.db</code> | <code>SportsDatabase.insert_*</code> | SQLite: videos + sport tables | <code>db.py:168-349</code> |

This architecture is clean, modular, and sports-agnostic. To add a new observability report, you'd follow the exact pattern used by `validate-delivery` (build summary ? render markdown/JSON ? write files), pulling data from the already-computed manifest rather than re-querying the DB.